4 Combinational Circuits-I

Introduction

- A combinational circuit produces an output as a function of input.
- The inputs and outputs are digital in nature, i.e., only '0' or '1'.
- It does not have any memory or feedback.
- So, the present value of outputs does not depend on the past value of output.
- Faster in speed as compared to sequential logic circuit.
- Easy to design using other gates.

Combinational Circuits

- Combinational circuits have interconnection of logic gates.
- Combinational circuits convert binary code into the desired output.
- A block diagram of a combinational circuit is shown below in the figure.

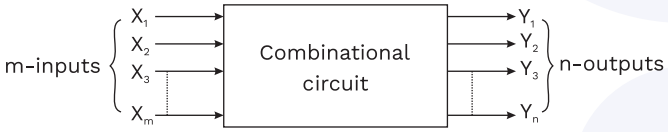


Fig. 4.1 Block Diagram of Combinational Circuit

Grey Matter Alert!

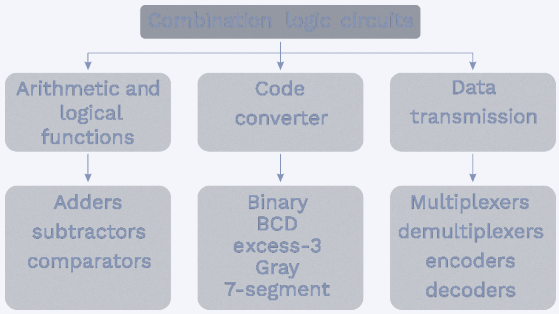


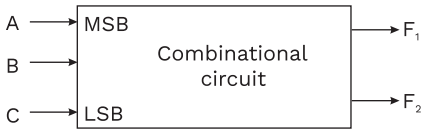
Fig. 4.2

To design any combinational circuit

- Consider that each output function is represented with reference to m-inputs.
- We have 'n' output functions (f_1, f_2, f_3 ... f_n).
- Write truth table for each function in terms of inputs.
- Convert the functions in terms of m-inputs using k-map.
- Draw the logic diagram.

Example 1:

Design a combinational circuit for functions F_1 and F_2 with the help of given truth table.



Truth table for this combinational circuit is

A	B	C	F_1	F_2
0	0	0	0	1
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	0
1	1	1	1	1

Table. 4.1

Solution:

For F_1 K-map

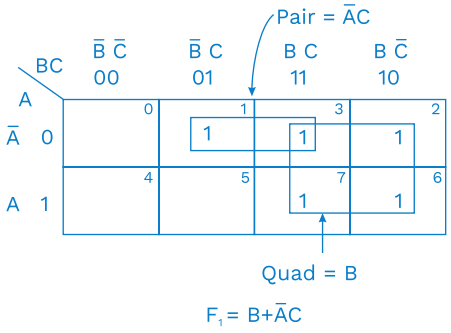
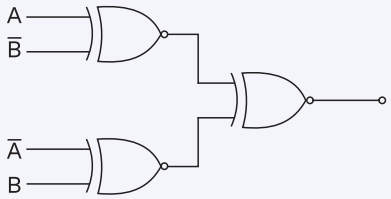


Fig. 4.3

Previous Years' Questions

The output of the circuit shown in the figure is equal to:

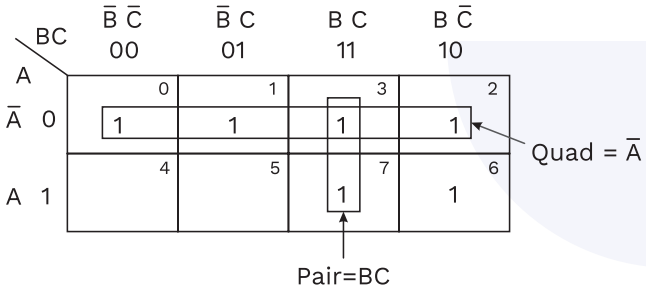


(A) $\overline{A}B + A\overline{B}$ (B) 0

(C) 1 (D) $\overline{(A \oplus B)} \oplus \overline{(A \oplus B)}$

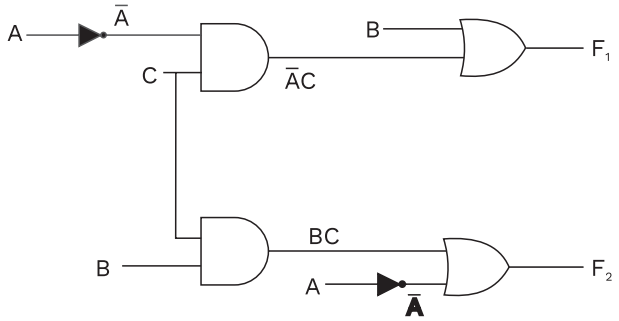
Solution: (C)

For F_2 K-map



$F_2 = \overline{A}+BC$

Logic diagram.



Rack Your Brain

To add two m-bit numbers, the required numbers of half adder are:

(A) 2m–1 (B) 2^m–1

(C) 2m+1 (D) 2m

Binary Adder

1. It is a type of combinational circuit that is used for the addition of binary bits.
2. Binary adders are of two types:
   a) Half adder
   b) Full adder

Half Adder

- It is used to add two binary bits.
- It has two binary inputs and two binary outputs.
- The input variables are called augend and addend bits.
- The output variables are called Sum (S) and Carry (C_0).

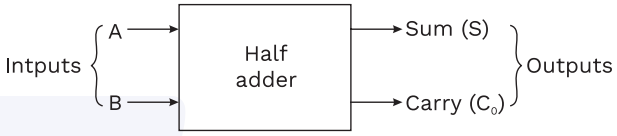


Fig. 4.4 Half Adder

Truth table:

Inputs		Outputs	
A	**B**	**S**	**C_0**
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table. 4.2

So, in half adder we get,

Sum(S) = $\overline{A}B + A\overline{B}$

$S = A \oplus B$

Carry (C_0) = AB

Logic diagram:

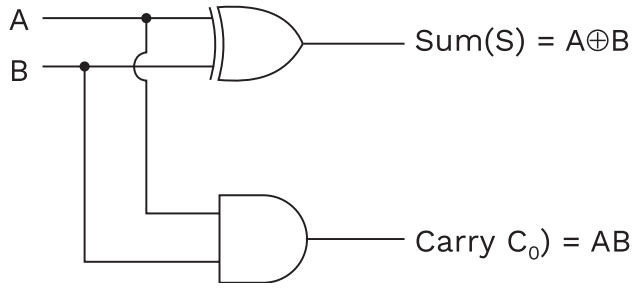
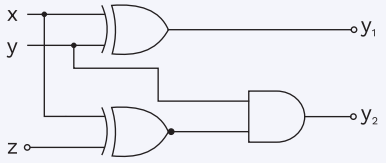


Fig. 4.5 Half Adder Using Logic Gate

Previous Years' Questions

The circuit shown below, is a controlled half adder/half subtractor. The inputs to half adder/half subtractor are x and y, while z is a control. The outputs are y_1 and y_2.



(A) Half adder for z = 0
(B) Half subtractor for z = 1
(C) Half adder for z = 1, and half subtractor for z = 0
(D) Half adder regardless of whether z = 0 or z = 1 due to design defect

Solution: (A), (B)

Half adder using NAND gate:

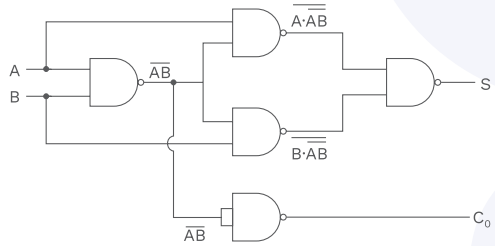


Fig. 4.6 Half Adder Using NAND Gate

$$S = \overline{A \cdot \overline{AB}} \cdot \overline{B \cdot \overline{AB}}$$

$$S = (A \cdot \overline{AB}) + (B \cdot \overline{AB})$$

$$S = A(\overline{A} + \overline{B}) + B(\overline{A} + \overline{B})$$

$$S = A\overline{A} + A\overline{B} + B\overline{A} + B\overline{B} = A\overline{B} + \overline{A}B$$

$$S = A \oplus B$$

$$C_0 = \overline{\overline{AB}}$$

$$C_0 = AB$$

Half adder using NOR gate:

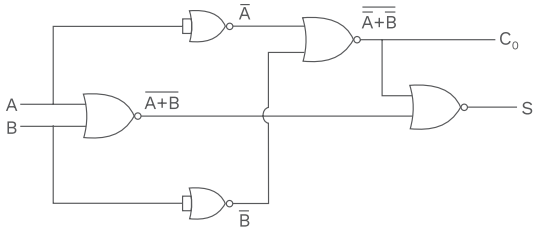


Fig. 4.7 Half Adder Using NOR Gate

$$C = \overline{\overline{A} + \overline{B}} = \overline{\overline{A}} \cdot \overline{\overline{B}} = A \cdot B$$

$$S = \overline{\overline{(A + B)} + \overline{(\overline{A} + \overline{B})}}$$

$$S = \overline{(A + B)} \cdot \overline{(\overline{A} + \overline{B})}$$

$$S = (A + B) \cdot (\overline{A} + \overline{B})$$

$$S = A\overline{A} + A\overline{B} + \overline{A}B + B \cdot \overline{B}$$

$$S = 0 + A\overline{B} + \overline{A}B + 0$$

$$S = A\overline{B} + \overline{A}B$$

$$S = A \oplus B$$

Full Adder

- Full adder is used to add 3 binary bits.
- It consists of 3 inputs and, 2 outputs.
- The inputs are A, B and C_{in}, and the outputs are sum (S) and carry (C_{out}) . A and B are significant bits to be added, and C_{in} is the carry from previous lower significant position.



Fig. 4.8 Full Adder

Rack Your Brain

What are the S (sum) and C (carry) output bits of a half adder with input A = 1 and B = 1?

	S	C
(A)	1	1
(B)	1	0
(C)	0	1
(D)	0	0

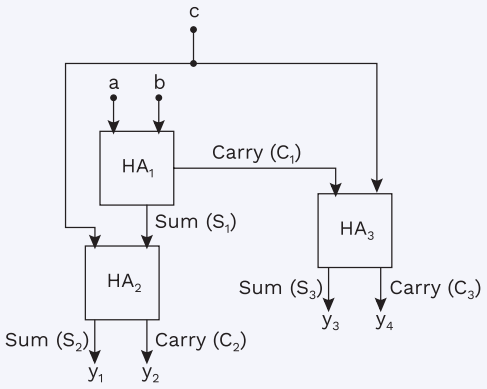
Truth table:

Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table. 4.3

Previous Years' Questions

Three half adders HA_1, HA_2 and HA_3 are inter-coupled as shown below. The four output functions y_1, y_2, y_3 and y_4 are expressed in terms of inputs a, b, and c. Which one of the following output expressions, is correct?



(A) $y_1 = (a \oplus b)c$ (B) $y_2 = (a \oplus b) \oplus c$
(C) $y_3 = ab \oplus c$ (D) $y_4 = a(b \oplus c)$
Solution: (C)

Expression for sum(S):

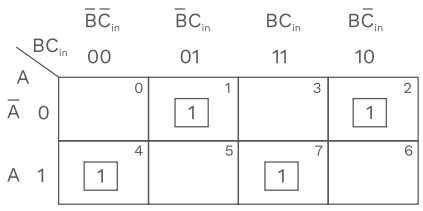


Fig. 4.9 Full Adder K-map for Sum

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

Expression for carry-out (C_{out}):

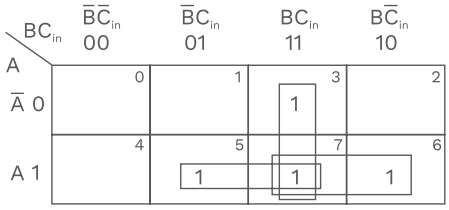


Fig. 4.10 Full Adder K-map for Carry Out

$$C_{out} = AB + BC_{in} + AC_{in}$$

Expression of sum(S):

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

$$= C_{in}(\bar{A}\bar{B} + AB) + \bar{C}_{in}(\bar{A}B + A\bar{B})$$

$$= C_{in}(A \odot B) + \bar{C}_{in}(A \oplus B)$$

$$= C_{in}\overline{(A \oplus B)} + \bar{C}_{in}(A \oplus B)$$

$$(\because A \odot B = \overline{A \oplus B})$$

Let $A \oplus B = X$,

$$S = C_{in}\bar{X} + \bar{C}_{in}X$$

$$S = C_{in} \oplus X \qquad (\because \bar{A}B + A\bar{B} = A \oplus B)$$

Rack Your Brain

If a full adder is designed with two half adders and OR gates, then a four-bit parallel adder without any initial carry requires:
(A) 8 half adders, 4 OR gates
(B) 8 half adders, 3 OR gates
(C) 7 half adders, 4 OR gates
(D) 7 half adders, 3 OR gates

Replace X by $A \oplus B$

$$S = C_{in} \oplus A \oplus B$$

$$S = A \oplus B \oplus C_{in}$$

Expression of Carry-Out (C_{out})

$$C_{out} = AB + BC_{in} + C_{in}A$$

We can rearrange this in terms of XOR gate

$$C_{out} = AB + (A \oplus B)C_{in}$$

Logic diagram:

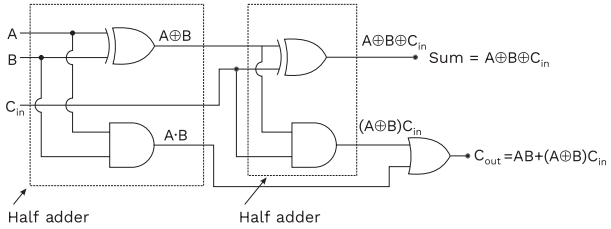


Fig. 4.11 Full Adder Using Logic Gate

Rack Your Brain

If a full adder is designed with two half adders and OR gates, then a four-bit parallel adder without any initial carry requires:

(A) 8 half adders, 4 OR gates
(B) 8 half adders, 3 OR gates
(C) 7 half adders, 4 OR gates
(D) 7 half adders, 3 OR gates

Half Subtractor

- Half subtractor is used for the subtraction of two binary bits.
- It consists of 2 inputs, i.e., minuend bit and subtrahend bit.
- It also consists of 2 outputs, i.e., difference (D) and borrow (B_0).

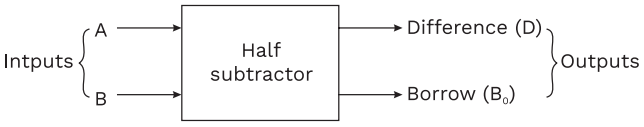


Fig. 4.12 Half Subtractor

Truth table:

Inputs		Outputs	
A	**B**	**D**	**B_0**
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Table. 4.4

from truth table, the expressions for difference and borrow are:

$$D = \bar{A}B + A\bar{B}$$
$$D = A \oplus B$$
$$B_0 = \bar{A}B$$

Logic diagram:

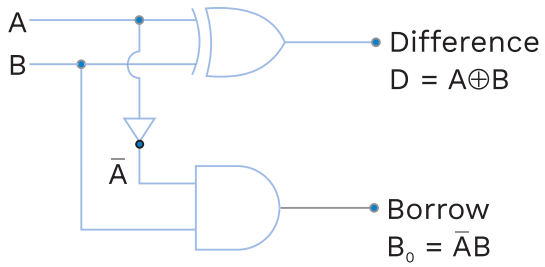


Fig. 4.13

Previous Years' Questions

For a binary half-subtractor having two inputs, A and B; the correct set of logical expressions for the outputs difference (D) and borrow (X) are:

(A) $D = AB + \bar{A}B, X = \bar{A}B$
(B) $D = \bar{A}B + A\bar{B} + AB, X = A\bar{B}$
(C) $D = \bar{A}B + A\bar{B}, X = \bar{A}B$
(D) $D = AB + \bar{A}\bar{B}, X = A\bar{B}$

Solution: (C)

Half subtractor using NAND gate:

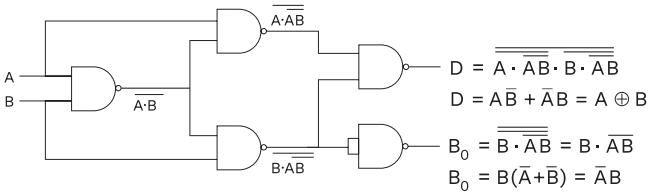


Fig. 4.14 Half Subtractor Using NAND Gate

Half subtractor using NOR gate:

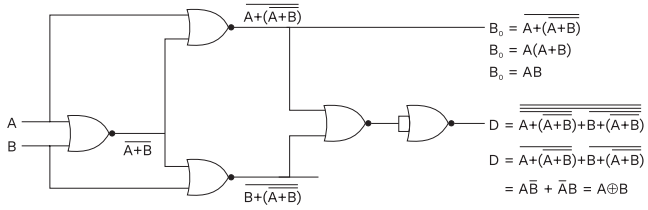
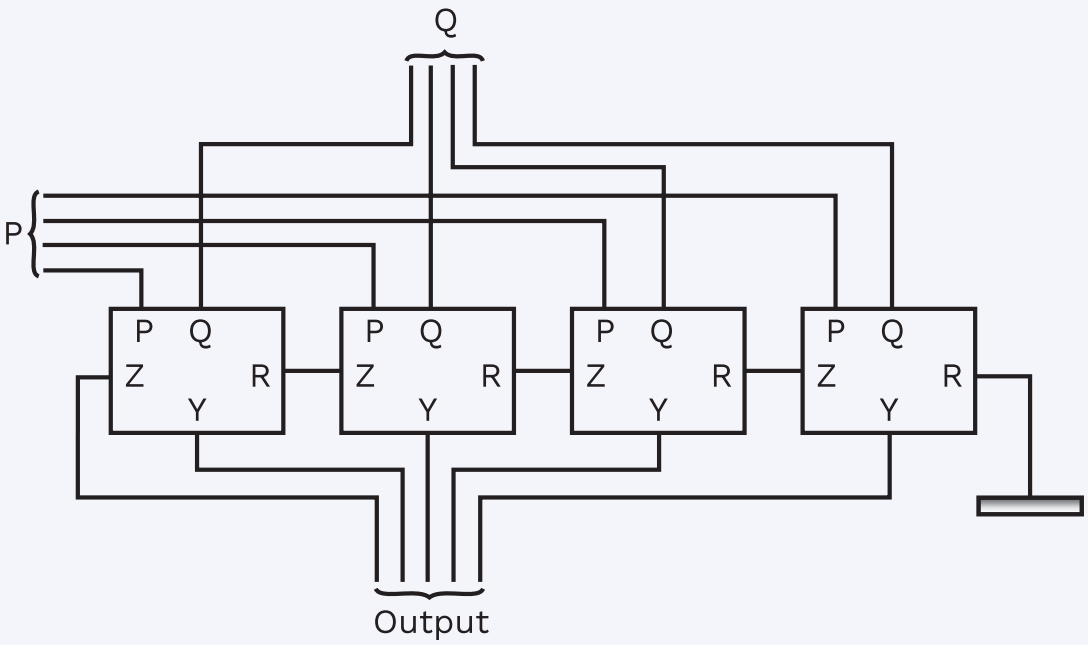


Fig. 4.15 Half Subtractor Using NOR Gate

Previous Years' Questions

The circuit shown in the figure has 4 boxes each described by inputs P, Q and R and outputs Y and Z with

$$Y = P \oplus Q \oplus R \qquad Z = RQ + \bar{P}R + Q\bar{P}$$



The circuit acts as a:

(A) 4 bit adder giving P+Q

(B) 4 bit subtractor giving P–Q

(C) 4 bit subtractor giving Q–P

(D) 4 bit adder giving P+Q+R

Solution: (B)

Full Subtractor

- Full subtractor is used for the subtraction of 3 binary bits.
- It consists of 3 inputs, i.e., A, B and B_{in} (borrow in).
- It consists of 2 outputs, i.e., difference (D) and borrow out (B_0).

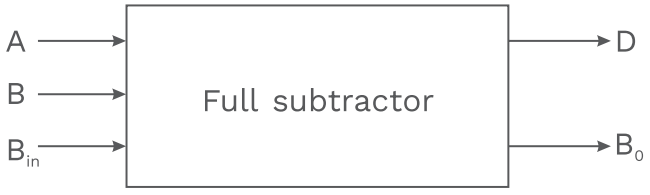


Fig. 4.16 Full Subtractor

Truth table:

Inputs			Outputs	
A	**B**	**B_{in}**	**D**	**B_0**
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Table. 4.5

Previous Years' Questions

Match list I with list II as shown below and select the correct answer using the codes given below the list.

List-I (circuit to be implemented)

P. 2-bit multiplier such that multiplicand $A = a_1a_0$, multiplier $B = b_1b_0$ and product $P = P_3P_2P_1P_0$

Q. Dual 4×1 MUX with common select lines

R. BCD adder subtractor

S. Two 4-bit multipliers

List-II (size of ROM)

I. $2^{10} \times 5$

II. $2^8 \times 8$

III. $2^9 \times 5$

IV. $2^{10} \times 2$

V. $2^4 \times 4$

	P	Q	R	S
(A)	IV	V	III	I
(B)	I	IV	II	III
(C)	V	IV	I	II
(D)	I	V	III	II

Solution: (D)

Expression for difference (D) and borrow out (B_0):

For difference (D),

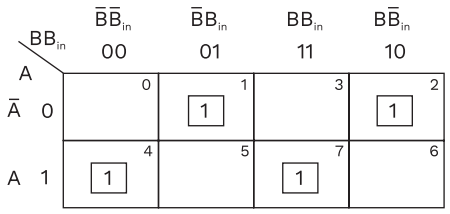


Fig. 4.17 Full Subtractor K-map for Difference

$$D = \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + A\bar{B}\bar{B}_{in} + ABB_{in}$$

$$D = B_{in}(\bar{A}\bar{B} + AB) + \bar{B}_{in}(\bar{A}B + A\bar{B})$$

$$D = B_{in}(A \odot B) + \bar{B}_{in}(A \oplus B) =$$

$$B_{in}\overline{(A \oplus B)} + \bar{B}_{in}(A \oplus B)$$

Let $A \oplus B = X$

$$D = B_{in}\bar{X} + \bar{B}_{in}X$$

$$D = B_{in} \oplus X$$

$$D = B_{in} \oplus (A \oplus B) = A \oplus B \oplus B_{in}$$

For borrow out (B_0),

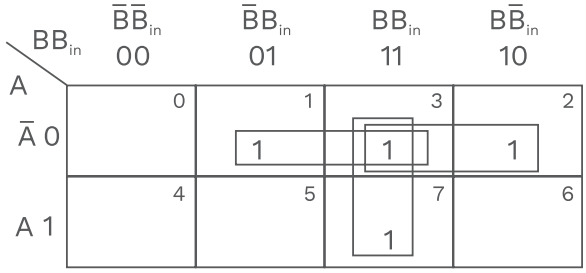


Fig. 4.18 Full Subtractor K-map for Borrow

$$B_0 = \bar{A}B_{in} + \bar{A}B + BB_{in}$$

$$B_0 = \bar{A}B + \bar{A}B_{in} + BB_{in}$$

We can re-arrange this in terms of Ex-OR gate.

$$B_0 = \bar{A}B + \overline{(A \oplus B)} \cdot B_{in}$$

Logic diagram:

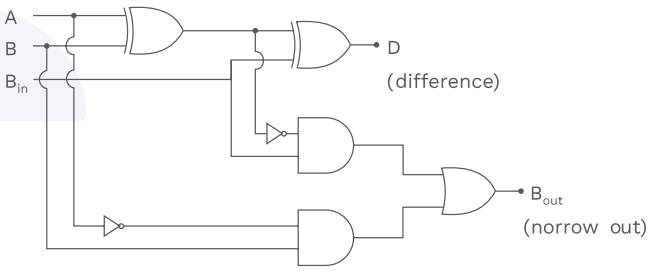


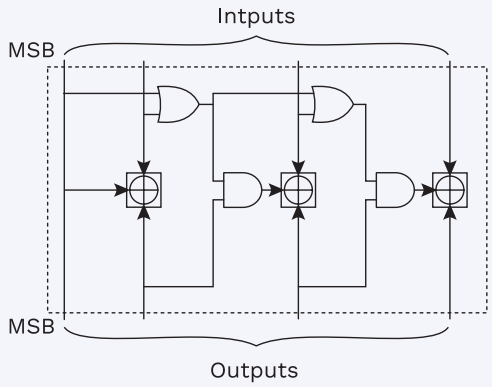
Fig. 4.19 Full Subtractor Using Logic Gate

Code Converters

- There are many binary codes that are used in digital systems. Some of them are as follows: BCD code, Excess-3 code, Gray code, etc.
- Sometimes, it is required to convert from one code to another code. Different codes are used for different types of specific applications.
- The code converters can be implemented using logic gates, multiplexers or demultiplexers.

Previous Years' Questions

The circuit shown in the figure converts



(A) BCD to binary code
(B) Binary to excess-3 code
(C) Excess-3 to Gray code
(D) Gray to Binary code

Solution: (D)

Binary to Gray Converter

Let 4 binary bits (input) are B_1, B_2, B_3 and B_4 and 4 gray code bits (output) are G_1, G_2, G_3 and G_4.

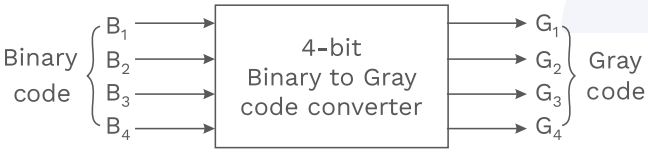


Fig. 4.20 Binary to Gray Converter

Truth table:

Inputs				Outputs			
Binary				Gray Code			
B_1	B_2	B_3	B_4	G_1	G_2	G_3	G_4
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

Table. 4.6

$G_1 = B_1$
$G_2 = B_1 \oplus B_2$
$G_3 = B_2 \oplus B_3$
$G_4 = B_3 \oplus B_4$

Expressions:

K-map for G_1: $G_1 = \sum m(8, 9, 10, 11, 12, 13, 14, 15)$

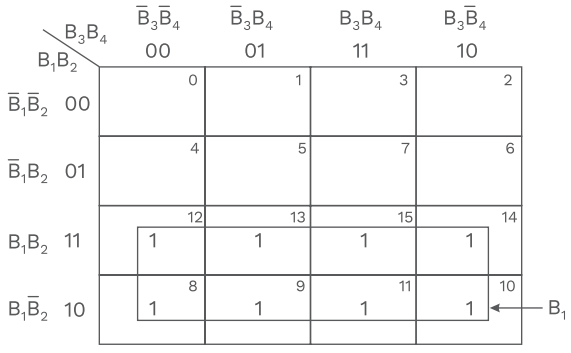


Fig. 4.21 K-map for G_1

$G_1 = B_1$

Rack Your Brain

Consider the statements that follow:

I. BCD to XS-3 code converter can be realised using only 4-bit parallel adder.

II. The BCD adder requires correction (adding 0110 to sum) if sum ±9 or carry is generated.

III. 1-Bit magnitude comparator can be realised using two NOT gates, two 2-input AND gates, and one 2-input X-NOR gate.

Of these, the incorrect statement(s) is (are)

(A) Only I (B) Only II

(C) Both I and II (D) None

K-map for G_2: $G_2 = \sum m(4, 5, 6, 7, 8, 9, 10, 11)$

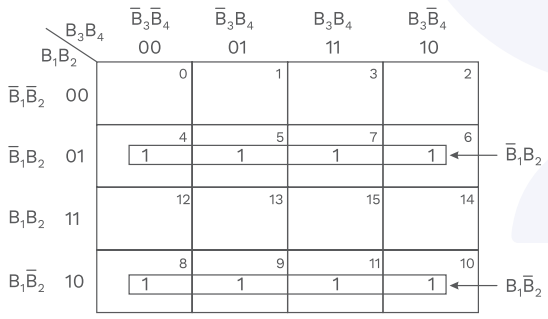


Fig. 4.22 K-map for G_2

$$G_2 = \bar{B}_1 B_2 + B_1 \bar{B}_2$$

$$G_2 = B_1 \oplus B_2$$

K-map for G_3: $G_3 = \sum m(2, 3, 4, 5, 10, 11, 12, 13)$

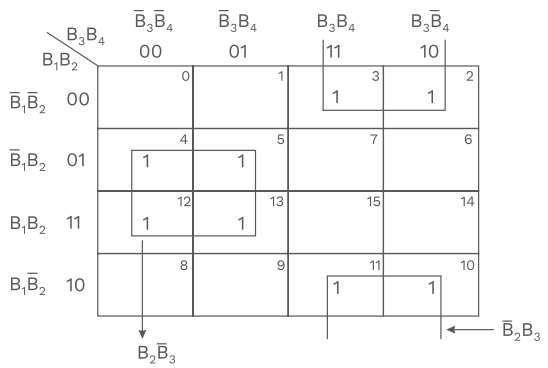


Fig. 4.23 K-map for G_3

$$G_3 = B_2\bar{B}_3 + \bar{B}_2 B_3$$

$$G_3 = B_2 \oplus B_3$$

K-map for G_4: $G_4 = \sum m(1, 2, 5, 6, 9, 10, 13, 14)$

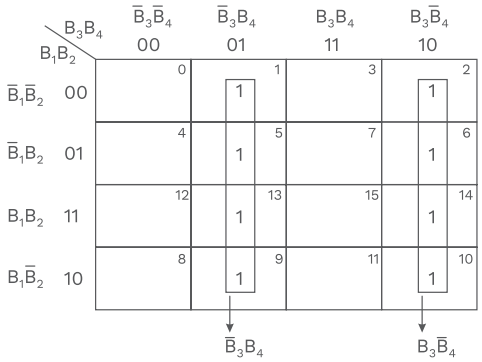


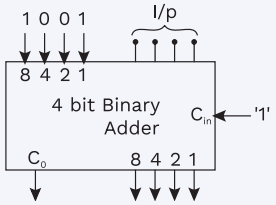
Fig. 4.24 K-map for G_4

$$G_4 = \bar{B}_3 B_4 + B_3 \bar{B}_4$$

$$G_4 = B_3 \oplus B_4$$

Previous Years' Questions

Which operation is possible to implement in the circuit below?



(A) Changing 1001 to 1100 and $C_{in} = 0$

(B) Changing 1001 to 1010 and $C_{in} = 0$

(C) Changing 1001 to 1101 and $C_{in} = 0$

(D) Changing 1001 to 1011 and $C_{in} = 0$

Solution: (D)

Logic diagram:

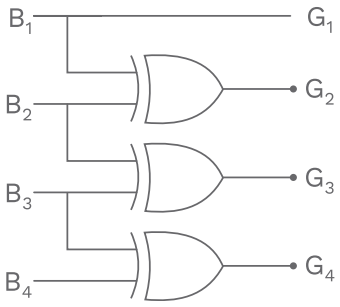
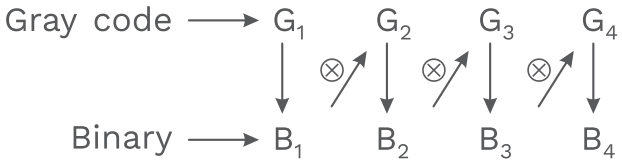


Fig. 4.25 Logic Diagram of Binary to Gray converter

Gray to Binary Converter



Fig. 4.26 Gray to Binary Converter



Truth table:

Inputs				Outputs			
Gray Code				Binary Code			
G_1	G_2	G_3	G_4	B_1	B_2	B_3	B_4
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
1	0	0	0	1	1	1	1
1	0	0	1	1	1	1	0
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	0	1	0	1	1
1	1	1	1	1	0	1	0

Table. 4.7

Expressions:

K-map for B_1: $B_1 = \sum m\ (8, 9, 10, 11, 12, 13, 14, 15)$

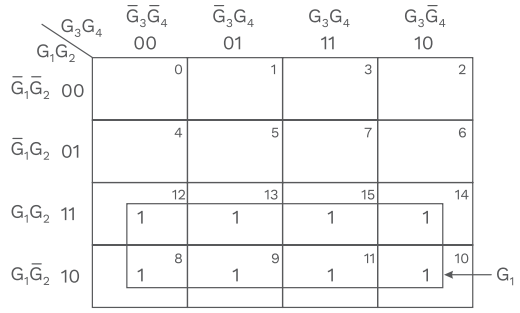


Fig. 4.27 K-map for B_1

Previous Years' Questions

The output Y of a 2-bit comparator is logic 1 whenever the 2-bit input A is greater than the 2-bit input B. The number of combination for which the output is logic 1, is:

(A) 4 (B) 6

(C) 8 (D) 10

Solution: (B)

$$B_1 = G_1$$

K-map for B_2: $B_2 = \sum m\ (4, 5, 6, 7, 8, 9, 10, 11)$

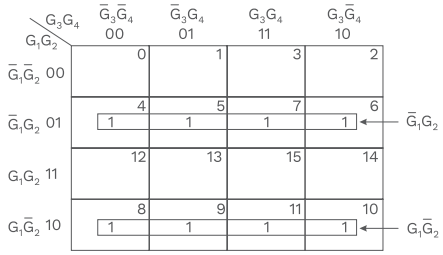


Fig. 4.28 K-map for B_2

$$B_2 = \bar{G}_1 G_2 + G_1 \bar{G}_2$$

$$B_2 = G_1 \oplus G_2$$

K-map for B_3: $B_3 = \sum m\ (2, 3, 4, 5, 8, 9, 14, 15)$

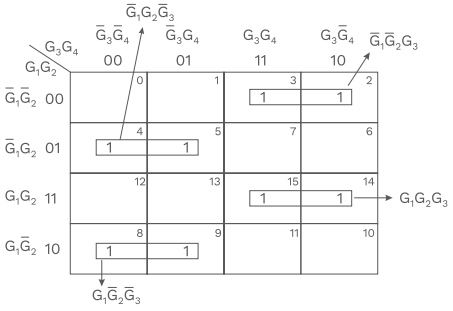


Fig. 4.29 K-map for B_3

$$B_3 = \bar{G}_1\bar{G}_2G_3 + G_1G_2G_3 + G_1\bar{G}_2\bar{G}_3 + \bar{G}_1G_2\bar{G}_3$$

$$B_3 = \bar{G}_1[\bar{G}_2G_3 + G_2\bar{G}_3] + G_1[G_2G_3 + \bar{G}_2\bar{G}_3]$$

$$B_3 = \bar{G}_1[G_2 \oplus G_3] + G_1[G_2 \odot G_3]$$

$$B_3 = \bar{G}_1[G_2 \oplus G_3] + G_1\overline{[(G_2 \oplus G_3)]}$$

$$B_3 = G_1 \oplus G_2 \oplus G_3$$

K-map for B_4: $B_4 = \sum m\ (1, 2, 4, 7, 8, 11, 13, 14)$

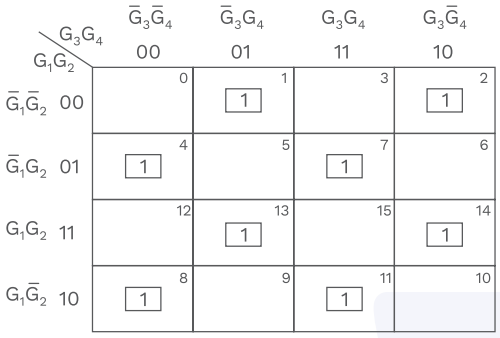
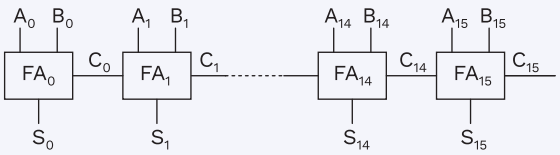


Fig. 4.30 K-map for B_4

$$B_4 = \bar{G}_1G_2\bar{G}_3\bar{G}_4 + G_1\bar{G}_2\bar{G}_3\bar{G}_4 + \bar{G}_1\bar{G}_2\bar{G}_3G_4 + G_1G_2\bar{G}_3G_4 + \bar{G}_1G_2G_3G_4 + G_1\bar{G}_2G_3G_4 + \bar{G}_1\bar{G}_2G_3\bar{G}_4 + G_1G_2G_3\bar{G}_4$$

Previous Years' Questions

A 16-bit ripple carry adder is realised using 16 identical full adders (FA) as shown in the figure. The carry-propagation delay of each FA is 12 ns, and the sum-propagation delay of each FA is 15 ns. The worst-case delay (in ns) of this 16-bit adder will be ___ .



Solution: 194.9- 195.1

After simplifying, we get

$$B_4 = G_1 \oplus G_2 \oplus G_3 \oplus G_4$$

Logic diagram:

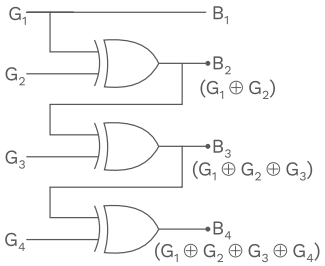


Fig. 4.31 Logic Diagram of Gray to Binary Converter

BCD to Excess-3 Code Converter

- It is used to convert the BCD code in to the Excess-3 code.
- We can obtain Excess-3 code by adding 3 in BCD code.

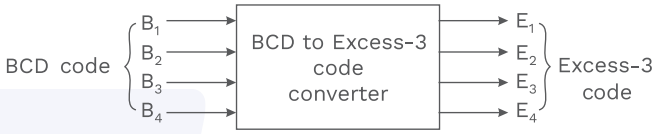


Fig. 4.32 BCD to Excess-3 Code Converter

Truth table:

Inputs				Outputs			
BCD code				Excess-3 Code			
B_1	B_2	B_3	B_4	E_1	E_2	E_3	E_4
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Table. 4.8

Since BCD is a valid code from 0 to 9.

Therefore, after 9, we use don't care (invalid states)

Don't care $\rightarrow \sum d(10, 11, 12, 13, 14, 15)$

K-map for E_1:

$E_1 = \sum m(5, 6, 7, 8, 9) + \sum d(10, 11, 12, 13, 14, 15)$

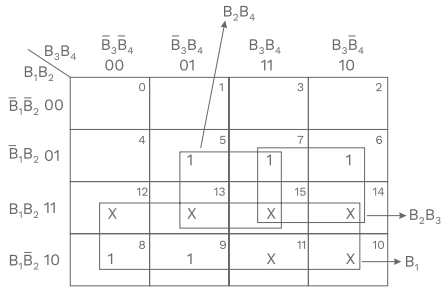


Fig. 4.33 K-map for E_1

$E_1 = B_1 + B_2B_3 + B_2B_4$

K-map for E_2:

$E_2 = \sum m(1, 2, 3, 4, 9) + \sum d(10, 11, 12, 13, 14, 15)$

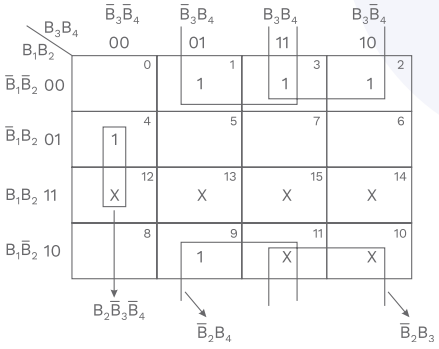


Fig. 4.34 K-mAP for E_2

$$E_2 = B_2\bar{B}_3\bar{B}_4 + \bar{B}_2B_3 + \bar{B}_2B_4$$

K-map for E_3:

$E_3 = \sum m(0, 3, 4, 7, 8) + \sum d(10, 11, 12, 13, 14, 15)$

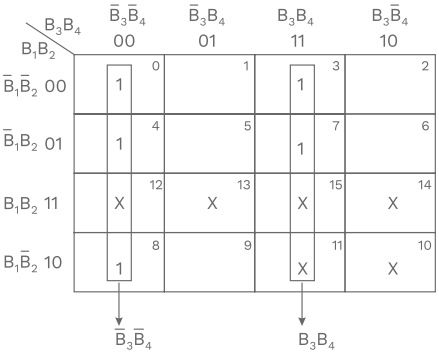


Fig. 4.35 K-map for E_3

$$E_3 = \bar{B}_3\bar{B}_4 + B_3B_4$$

K-map for E_4:

$E_4 = \sum m(0, 2, 4, 6, 8) + \sum d(10, 11, 12, 13, 14, 15)$

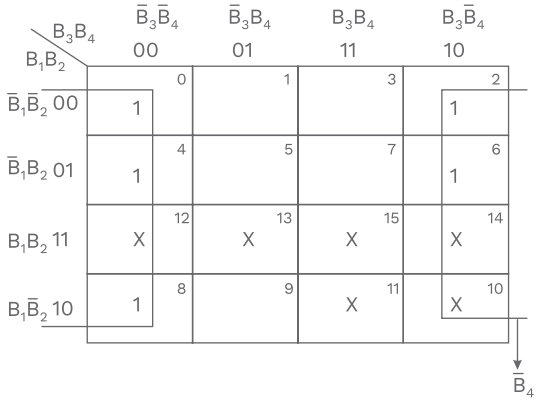


Fig. 4.36 K-map for E_4

$$E_4 = \bar{B}_4$$

Logic diagram:

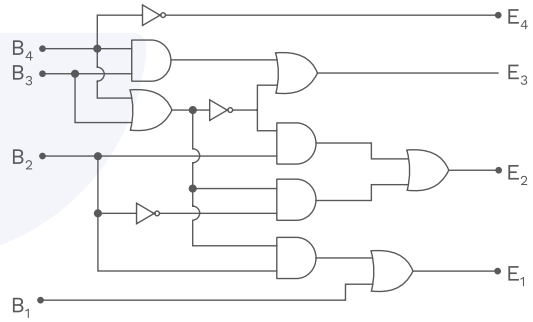


Fig. 4.37 Logic Diagram Of BCD to Excess-3 Code

BCD-to-seven Segment Converter

- It is used in the display devices.
- It provides outputs in order to energise the seven-segment display.
- The seven-segment display is used to display decimal number as outputs.
- The seven-segment display has seven light emitting segments.
- These segments are designed by letters 'a' to 'g'.

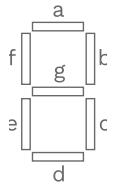


Fig. 4.38 Seven Segment Display

Number	Display	Segment Activated
0	0	a, b, c, d, e, f
1	1	b, c
2	2	a, b, d, e, g
3	3	a, b, c, d, g
4	4	b, c, f, g
5	5	a, c, d, f, g
6	6	a, c, d, e, f, g
7	7	a, b, c
8	8	a, b, c, d, e, f, g
9	9	a, b, c, d, f, g

Table. 4.9

Block diagram:

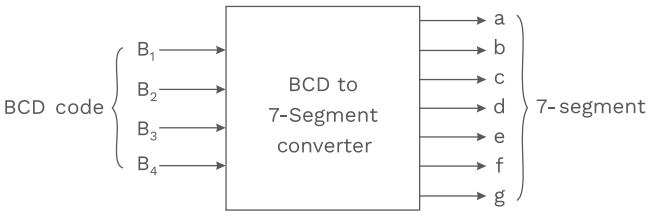


Fig. 4.39 BCD to 7-Segment Converter

Truth Table:

Inputs				Outputs						
BCD Code				7-Segment						
B_1	B_2	B_3	B_4	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

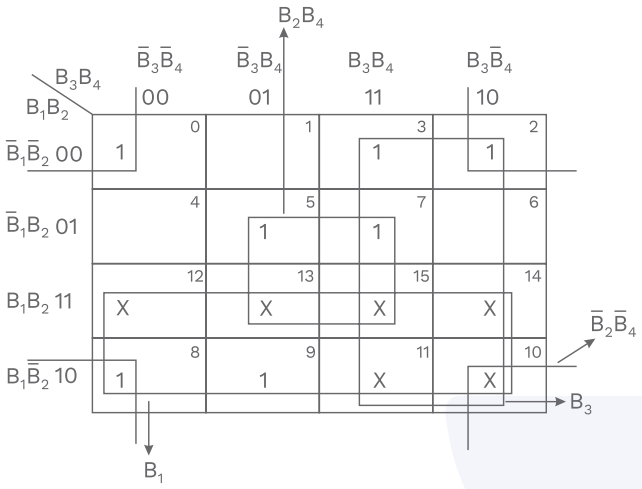
Table. 4.10

Expressions:

Here, we considered don't care, i.e., ∑d(10, 11, 12, 13, 14, 15)

For a:

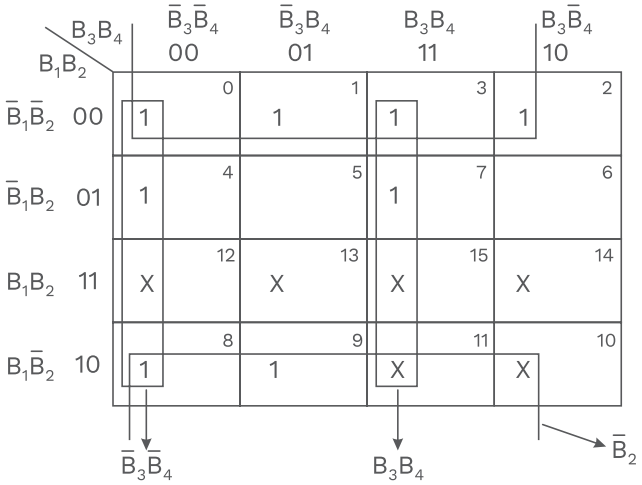
a = ∑m(0, 2, 3, 5, 6, 7, 8, 9)+∑d(10, 11, 12, 13, 14, 15)



$$a = B_1 + B_3 + B_2B_4 + \bar{B}_2\bar{B}_4$$

For b:

b = ∑m(0, 1, 2, 3, 4, 7, 8, 9) + ∑d(10, 11, 12, 13, 14, 15)



$$b = \bar{B}_2 + \bar{B}_3\bar{B}_4 + B_3B_4$$

For c:

c = ∑m(0, 1, 3, 4, 5, 6, 7, 8, 9) + ∑d(10, 11, 12, 13, 14, 15)

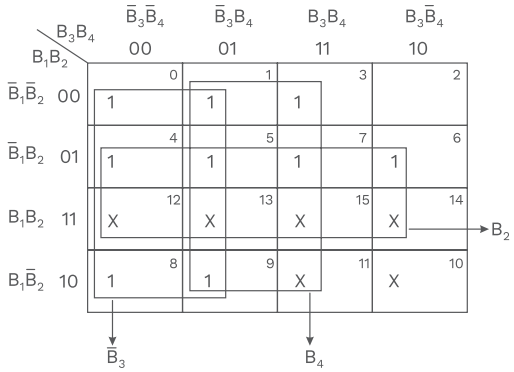


Fig. 4.40 K-map for c

$$c = B_2 + \bar{B}_3 + B_4$$

For d:

d = ∑m(0, 2, 3, 5, 6, 8, 9) + ∑d(10, 11, 12, 13, 14, 15)

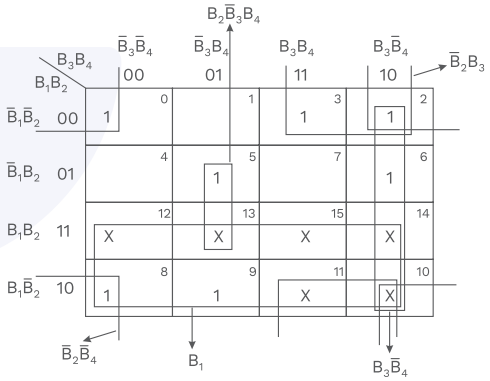


Fig. 4.41 K-map for d

$$d = B_1 + B_2\bar{B}_3B_4 + B_3\bar{B}_4 + \bar{B}_2B_3 + \bar{B}_2\bar{B}_4$$

For e:

e = ∑m(0, 2, 6, 8) + ∑d(10, 11, 12, 13, 14, 15)

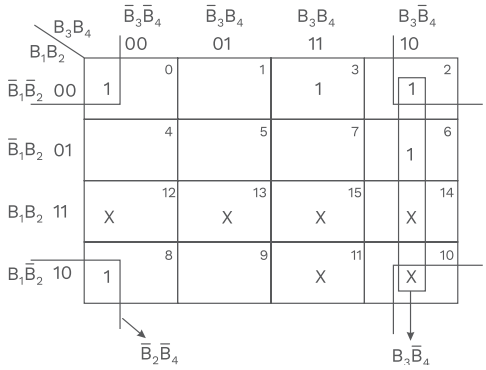


Fig. 4.42 K-map for e

$$e = B_3\bar{B}_4 + \bar{B}_2\bar{B}_4$$

For f:

$f = \sum m(0, 4, 5, 6, 8, 9) + \sum d(10, 11, 12, 13, 14, 15)$

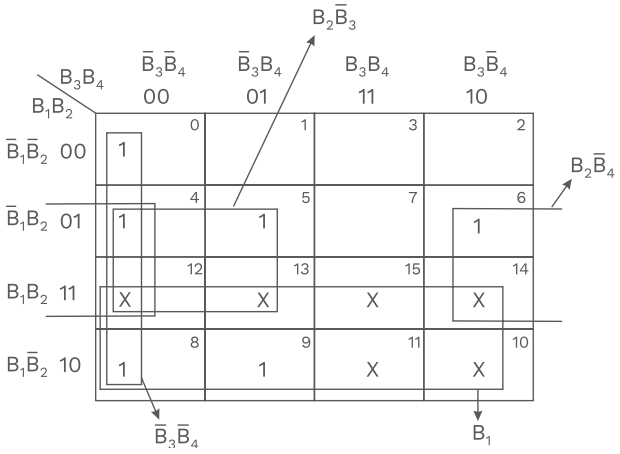


Fig. 4.43 K-map for f

$$\therefore = B_1 + \bar{B}_3\bar{B}_4 + B_2\bar{B}_4 + B_2\bar{B}_3$$

For g:

$g = \sum m(2, 3, 4, 5, 6, 8, 9) + \sum d(10, 11, 12, 13, 14, 15)$

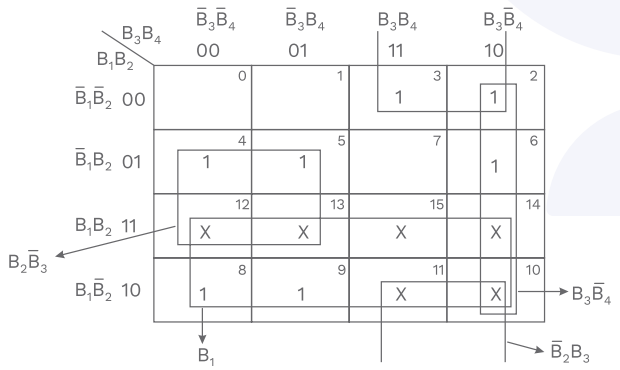


Fig. 4.44 K-map for g

$g = B_1 + \bar{B}_2B_3 + B_2\bar{B}_3 + B_3\bar{B}_4$

So, seven-segment outputs are:

$a = B_1 + B_3 + B_2B_4 + \bar{B}_2\bar{B}_4$

$b = \bar{B}_2 + \bar{B}_3\bar{B}_4 + B_3B_4$

$c = B_2 + \bar{B}_3 + B_4$

$d = B_1 + B_2\bar{B}_3B_4 + B_3\bar{B}_4 + \bar{B}_2B_3 + \bar{B}_2\bar{B}_4$

$e = B_3\bar{B}_4 + \bar{B}_2\bar{B}_4$

$f = B_1 + \bar{B}_3\bar{B}_4 + B_2\bar{B}_4 + B_2\bar{B}_3$

$g = B_1 + \bar{B}_2B_3 + B_2\bar{B}_3 + B_3\bar{B}_4$

Binary Comparators

- Binary comparators are used to compare the magnitude of two n-bit binary numbers.
- The possible outputs of a comparator are 1st input is less than 2nd input, (i.e., A < B), 1st input is equal to 2nd input, (i.e., A = B) and 1st input is greater than 2nd input, (i.e., A > B).



Fig. 4.45 n-bit Binary Comparator

1-Bit Binary Comparator

1-Bit binary comparator is used to compare the magnitude of two 1-bit numbers. It has three possible outputs (A<B), (A=B) and (A>B).

Block diagram:



Fig. 4.46 1-Bit Binary Comparator

Truth table:

Inputs		Outputs		
A	**B**	**A < B**	**A = B**	**A > B**
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

Table. 4.11

Expressions:

$\text{Output}(A < B) = \bar{A}B$

$\text{Output}(A = B) = \bar{A}\bar{B} + AB = A \odot B$

$\text{Output}(A > B) = A\bar{B}$

Logic diagram:

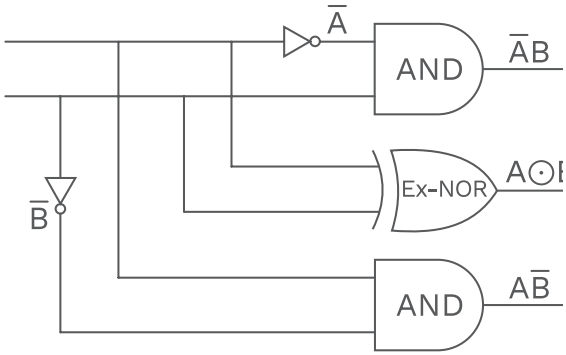


Fig. 4.47 Logic Diagram of Binary Comparator

2-Bit Binary Comparator

- 2-Bit binary comparator is used to compare the magnitude of 2-bit binary numbers.
- It has three possible outputs (A < B), (A = B) and (A > B).



Fig. 4.48 2-bit Comparator

Truth table:

Inputs				Outputs		
A		B		A < B	A = B	A > B
A_1	A_2	B_1	B_1			
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

Table. 4.12

Expressions:

Output (A < B) = ∑m (1, 2, 3, 6, 7, 11)

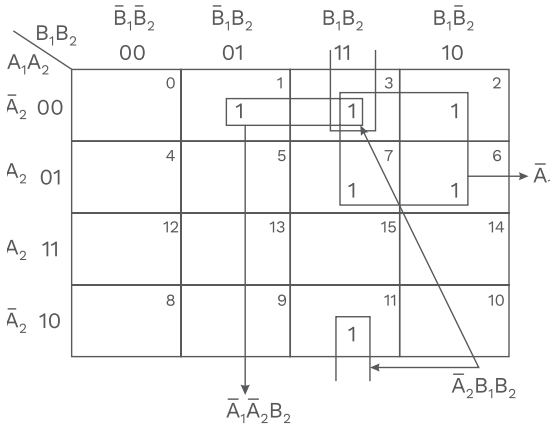


Fig. 4.49 K-map for A<B

∴ Output (A < B) = $\bar{A}_1B_1 + \bar{A}_2B_1B_2 + \bar{A}_1\bar{A}_2B_2$

Output (A > B) = ∑m(4, 8, 9, 12, 13, 14)

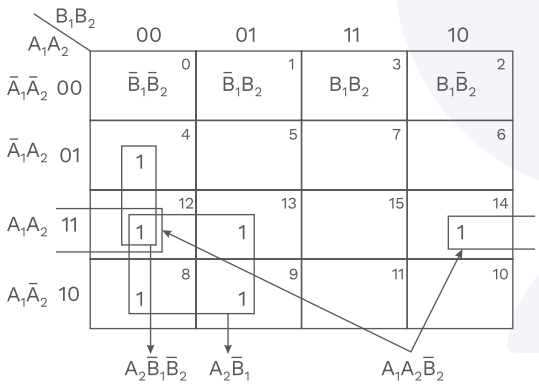


Fig. 4.50 K-map for A>B

∴ Output (A > B) = $A_2\bar{B}_1\bar{B}_2 + A_1\bar{B}_1 + A_1A_2\bar{B}_2$

Output (A = B) = ∑m(0, 5, 10, 15)

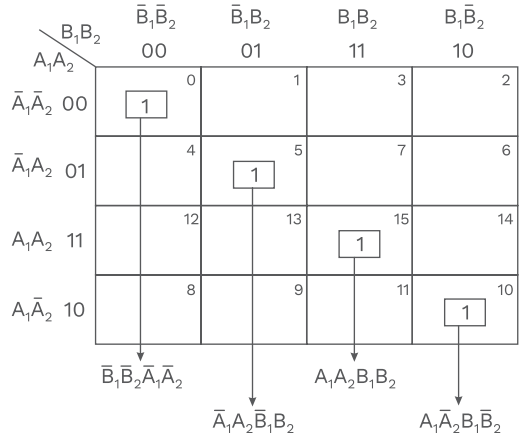


Fig. 4.51 K-map for A=B

Output (A = B) = $\bar{A}_1\bar{A}_2\bar{B}_1\bar{B}_2 + \bar{A}_1A_2\bar{B}_1B_2 + A_1\bar{A}_2B_1\bar{B}_2 + A_1A_2B_1B_2$

$= \bar{A}_1\bar{B}_1(\bar{A}_2\bar{B}_2 + A_2B_2) + A_1B_1(\bar{A}_2\bar{B}_2 + A_2B_2)$

$= \bar{A}_1\bar{B}_1(A_2 \odot B_2) + A_1B_1(A_2 \odot B_2)$

$= (A_2 \odot B_2)(\bar{A}_1\bar{B}_1 + A_1B_1)$

$= (A_2 \odot B_2)(A_1 \odot B_1)$

∴ Output (A = B) = $(A_1 \odot B_1)(A_2 \odot B_2)$

Logic diagram:

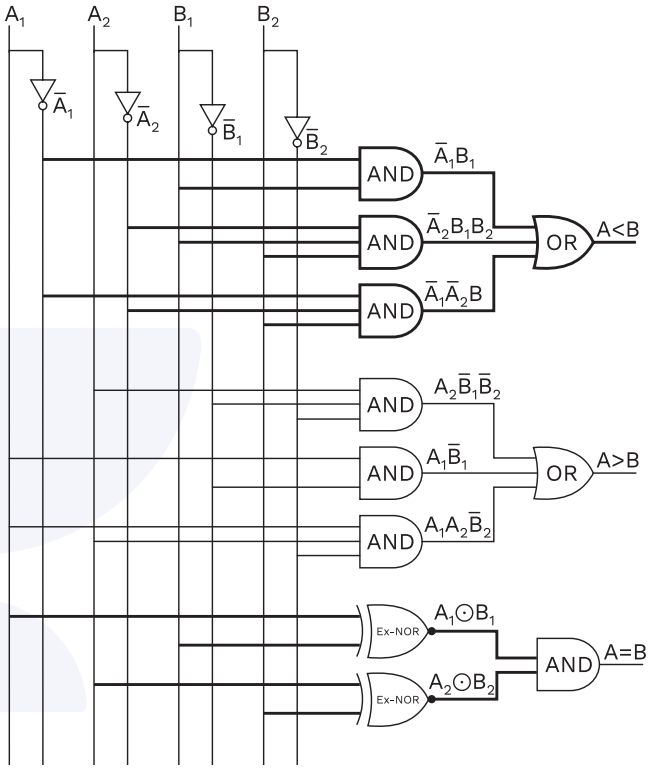


Fig. 4.52 Logic Diagram of Comparator

Parallel Binary Adder

- A digital circuit that adds 2 binary numbers in parallel form and produces the arithmetic sum bits as parallel outputs are called as binary parallel adder.
- It comprises full adders that are connected in cascade form/chain.
- Each full adder's output carry is used as the input carry of the next full adder in the chain.
- In parallel adder, only full adders are used due to cascading of adders.
- The addition of least significant bits can also be done using half adder.

- A full adder with carry-in (C_{in}) taken as grounded, (i.e., $C_{in} = 0$) , works as a half-adder.
- Two n-bit binary numbers can be added together using a parallel adder.
- The number of full adders is equal to the number of bits in each number of addition.

4-Bit Parallel Adder

- It is used for adding two 4-bit binary numbers.
- Required number of full adders is 4.
- Let two binary numbers are A and B.

$$A \rightarrow A_4\ A_3\ A_2\ A_1$$

$$B \rightarrow B_4\ B_3\ B_2\ B_1$$

$$Sum \rightarrow S_4\ S_3\ S_2\ S_1$$

$$Carry\text{-}Out \rightarrow C_4\ C_3\ C_2\ C_1$$

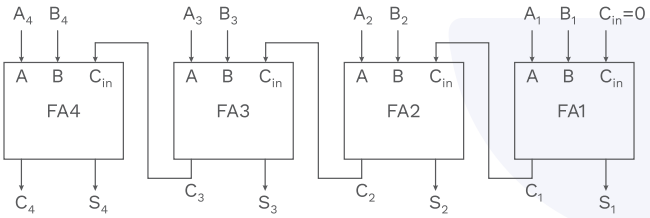


Fig. 4.53 4-Bit Parallel Adder

For example, if $A \rightarrow A_4\ A_3A_2A_1 = 1\ 1\ 1\ 1$

$$B \rightarrow B_4\ B_3B_2B_1 = 0\ 1\ 1\ 1$$

Full adder 1 $\rightarrow S_1 = A_1 + B_1 + C_{in} = 1 + 1 + 0 = 0$, $C_1 = 1$

Full adder 2 $\rightarrow S_2 = A_2 + B_2 + C_1 = 1 + 1 + 1 = 1$, $C_2 = 1$

Full adder 3 $\rightarrow S_3 = A_3 + B_3 + C_2 = 1 + 1 + 1 = 1$, $C_3 = 1$

Full adder 4 $\rightarrow S_4 = A_4 + B_4 + C_3 = 1 + 0 + 1 = 0$, $C_4 = 1$

We have,

$Sum \rightarrow S = S_4S_3S_2S_1 = 0\ 1\ 1\ 0$

$Carry\text{-}out \rightarrow C_4 = 1$

Note:
By cascading two 4-bit parallel adders, we can design an 8-bit parallel adder. One 4-bit adder for lower 4-bits and one 4-bit adder for higher 4-bits to add the 8-bit inputs are needed. Such adders are called cascading parallel adders.

Ripple Carry Adder

- A parallel binary adder is also known as a ripple carry adder.
- The parallel adder is designed in such a manner that each stage's carry-out is connected to the next stage's carry-in.
- The sum and carry out is generated after the carry-in of the full adder occur.
- In this adder, the final output produced only after carry-out is generated by each full adder stage and forwarded (rippled) to its next stage.
- We get a result after some time delay (propagation delay).

Previous Years' Questions

A 16-bit parallel adder is designed using 16, 1-bit full adders. Each 1-bit full adder is designed using X-OR, AND and OR gates. The delay contributed by X-OR gate is 20ns, while that contributed by AND/OR gate is 10 ns. The number of additions per second that a 16-bit parallel adder can perform reliably, will be:

(A) 2.94×10^6 (B) 3.125×10^6
(C) 3.33×10^6 (D) 4×10^6

Solution:

Carry Look-Ahead Adder

- In the parallel adder/ripple carry adder, there was a considerable time delay to get valid result.
- A look ahead carry is a technique to reduce propagation delay.
- It can produce results as faster as compared to parallel adder.
- To minimise the delay, it requires some additional hardware which could make it complex.
- It has two new binary variables for carry propagation and carry generation.

Previous Years' Questions

In a 4-bit carry look ahead adder, the X-OR gate has a delay of 10ns, and AND/OR gate has, delay of 5ns each. The sum will appear after delay:

(A) 35 ns (B) 30 ns

(C) 25 ns (D) 20 ns

Solution: (B)

Carry propagate:

Let two numbers A, B and carry input (C_{in}).

$C_{in} \rightarrow 0$	1	0	1
$A \rightarrow 0$	0	1	1
$B \rightarrow 1$	1	0	0
0 1	1 0	0 1	1 0

Carry-out

Here, carry-out is depending on carry-in (C_{in}), that's why it is called carry propagate. It is denoted by P_i.

Since any one of the input is 1, therefore

$$P_i = A \oplus B$$

Carry generate:

To understand it, let us consider,

C_{in}	0	1
A	1	1
B	1	1
	1 0	1 1

Carry-out

Here, carry-out does not depend on carry-in. Regardless of the value of C1n, whether it is 0 or 1, it always generates carry-out. Since both, the inputs are 1, therefore G_i = AB. The full adder circuits are redrawn as shown in figure:

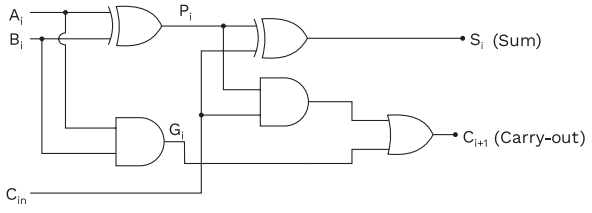


Fig. 4.54 Full Adder

Here the subscript i indicates the stage of adders.

$C_i \rightarrow$ Input carry

$P_i \rightarrow$ Carry propagate

$G_i \rightarrow$ Carry generate

We can write the expression of P_i and G_i from above circuit,

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

The sum and carry-out can be represents as:

$$S_i = P_i \oplus C_i$$

$$S_i = A_i \oplus B_i \oplus C_i$$

and $C_{out} = C_{i+1} = G_i + P_i C_i$

where C_{i+1} = next stage carry

Let 4-bit carry look ahead adder with input carry $C_{in} = C_0$

$$C_{i+1} = G_i + P_i C_i \quad \text{...(i)}$$

For i = 0

$$C_1 = G_0 + P_0 C_0 \quad \text{...(ii)}$$

For i = 1

$$C_2 = G_1 + P_1 C_1$$

$$C_2 = G_1 + P_1 (G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0 \quad \text{...(iii)}$$

For i = 2

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_0)$$

$$= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \quad \text{...(iv)}$$

For i = 3

$$C_4 = G_3 + P_3 C_3$$

$$= G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0)$$

$$= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 \quad \text{...(v)}$$

Logic diagram:

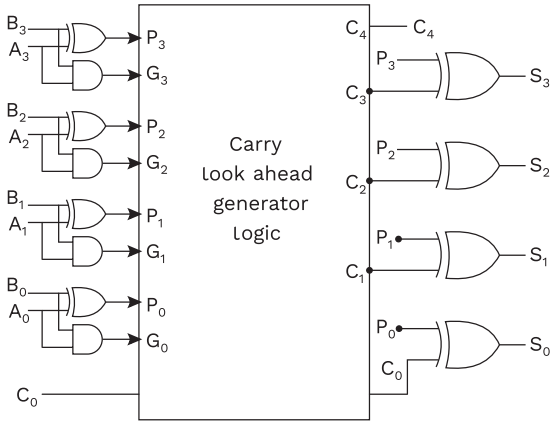


Fig. 4.55 Carry Look Ahead Generator

$A \rightarrow A_3A_2A_1A_0$

$B \rightarrow B_3B_2B_1B_0$

Here, $G_i = A_i \cdot B_i$

$P_i = A_i \oplus B_i$

So, $G_1 = A_1 \cdot B_1$

$P_1 = A_1 \oplus B_1$

$G_2 = A_2 \cdot B_2$

$P_2 = A_2 \oplus B_2$

$G_3 = A_3 \cdot B_3$

$P_3 = A_3 \oplus B_3$

So, we can directly obtain the final sum S_3 without any delay.

$S_3 = P_3 \oplus C_3$

$S_3 = A_3 \oplus B_3 \oplus C_3$

$C_{out} = C_4$

Rack Your Brain

The sum of ripple carry adder is:

(A) $S_i = A_i \oplus B_i \oplus C_i$

(B) $S_i = A_i B_i + A_i C_i + B_i C_i$

(C) $S_i = A_i + B_i + C_i$

(D) $S_i = A_i B_i C_i$

Parallel Subtractor/Binary Subtractor

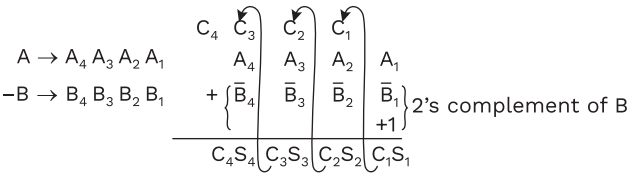
The subtraction of two numbers A and B can also be done by taking 2's complement of subtrahend and adding it to the minuend.

Subtraction : A – B

↑ ↑

Minuend Subtrahend

A – B = A + (2's complement of B)



A–B can be performed with adders consisting of inverters between each data input B of the full adder.

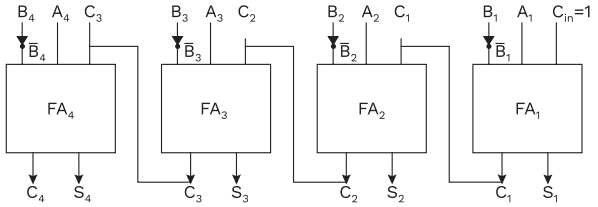


Fig. 4.56 Parallel Subtractor

- To perform a subtraction, input carry must be 1.
- 1's complement can be designed with inverters (not gate).

4-Bit Parallel Adder/Subtractor

- It is also called an adder cum subtractor.
- Difference = A+(2's complement of B) = A + [(1's complement of B)+1]
- The circuit diagram of adder cum subtractor is shown in the figure.

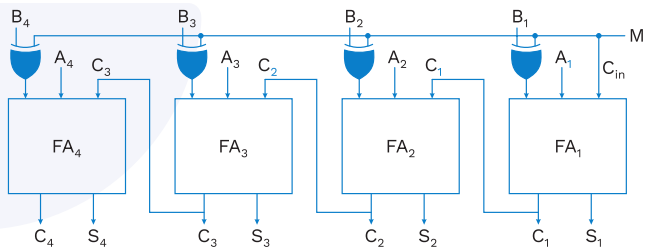


Fig. 4.57 4-Bit Parallel Adder/Subtractor

Case 1: When control input (M) = 0 and carry-in (C_{in}=0)

Let 4-bit binary numbers

$A = A_4\ A_3\ A_2\ A_1$

$B = B_4\ B_3\ B_2\ B_1$

Output of Ex-OR gates are B_1, B_2, B_3 and B_4.

$\because \begin{bmatrix} 0 \oplus 0 = 0 \\ 0 \oplus 1 = 1 \end{bmatrix}$, i.e., $0 \oplus B = B$

This circuit works as a 4-bit parallel adder, when control input (M) = 0

It performs the addition operation, i.e., A+B.

Case 2: When control input, M = 1 and carry-in, $C_{in} = 1$

Output of Ex-OR gates are $\bar{B}_1, \bar{B}_2, \bar{B}_3$ and $\bar{B}_4$, (i.e., 1's complement of B).

$\because \begin{bmatrix} 1 \oplus 0 = 1 \\ 1 \oplus 1 = 0 \end{bmatrix}$ i.e. $1 \oplus B = \overline{B}$

We know C_{in} is also 1, that will add with $\overline{B}$, (i.e., 2's complement of B). Now, this circuit performs the operation A+2's complement of B, (i.e., A–B operation). Hence, this circuit acts as a 4-bit parallel subtractor.

Bcd Adder

- It is also called a decimal adder.
- It is used in computers and calculators that excute arithmetic operations in decimal numbers directly.
- It accepts binary coded form of decimal numbers as inputs and produce the results also in decimal numbers.
- 4-bit adders are required to code each decimal digit and 1-bit for carry.
- We know that in BCD addition, if a carry is generated or the sum exceeds 9, then we need a correction, i.e., a correction by adding 6.
- We need to design that correction circuit first.
- If sum (S_4 S_3 S_2 S_1) is greater than 9, then output (Y) is taken as 1.
- This output (Y) is used to detect the error in the sum, which is caused by either sum is greater than 9 or a carry-out is generated in sum.

Sum				Outputs
S_4	S_3	S_2	S_1	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Table. 4.13

Previous Years' Questions

When a binary adder is used as BCD adder, the sum is:
(A) Correct when it is < 9
(B) Correct when it is > 9
(C) Correct when it is < 16
(D) None of these
Solution: (B)

The expression to detect sum is greater than 9 is:

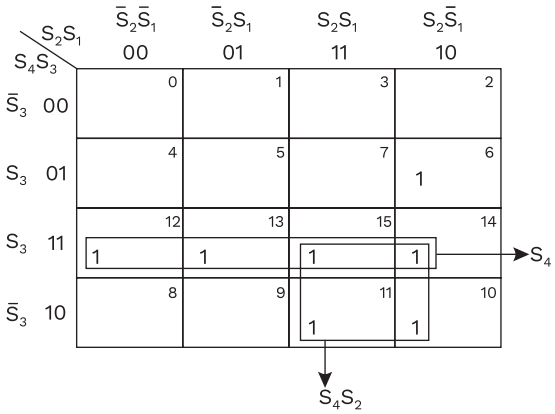


Fig. 4.58 K-map for BCD Adder

$$Y = S_4S_3 + S_4S_2$$

Circuit diagram of BCD adder:

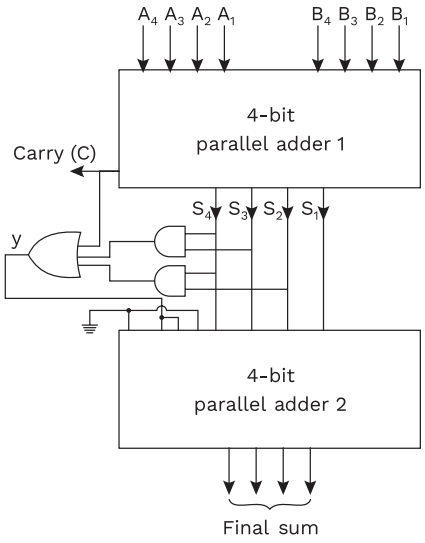


Fig. 4.59 BCD Adder

If the sum (S_4 S_3 S_2 S_1) is greater than 9 or carry-out (C) is 1, then we need to add 6 in the generated sum (S_4 S_3 S_2 S_1) to get the correct sum (final sum).

For example: Let the generated sum, S_4 S_3 S_2 S_1 = 1010 and carry-out C = 0

$$Y = S_4S_3 + S_4S_2 + C$$

$$Y = 0 + 1 + 0 = 1 + 0 = 1$$

In parallel adder 2, one input is S_4 S_3 S_2 S_1 and other one is from Y, (i.e., 0110). So, we get the correct sum.

Now let carry-out C = 1 and sum $S_4S_3S_2S_1$ = 1001

$$Y = S_4S_3 + S_4S_2 + C$$

$$Y = 0 + 0 + 1$$

$$Y = 1$$

In parallel adder 2, one input is S_4 S_3 S_2 S_1 and another is 0110.

So we get the correct sum.

Rack Your Brain

How many don't care inputs are there in a BCD adder?

(A) 312 (B) 128
(C) 256 (D) 200

Bcd Subtractor

- In BCD subtraction, 9's complement of the subtrahend is added to the minuend.
- We can obtain 9's complement of a number by subtracting that number from 9.
- If the result is invalid BCD, then correct the result by adding 6.
- After the addition of 6 if carry generated, we add the carry into the result.

Example 2:

BCD subtraction: 9 – 4

Solution:

9 – 4

Minuend Subtrahend

9 = 1001 → BCD

9's complement of 4 = 9–4 = 5

5 = 0101 → BCD

Now,

9 = 1001
5 = +0101
1110 → Invalid BCD
0110 → Add 6
10100
↳ + 1 → Add end around carry
0101 = +5 → Valid BCD

Hence 9 – 4 = 5

Previous Years' Questions

In order to get 9's complement of a BCD data, which one of the following should be added to its 1's complement?

(A) 1010 (B) 1000
(C) 1100 (D) 0110

Solution: (D)

- A combinational logic circuit is a circuit whose outputs depend only on the present state of inputs, i.e., each output is a function of the inputs.
- Logic gates are the simplest example of combinational circuits.

Chapter Summary

- A half adder is used for the addition of two binary bits.
- A full adder is an arithmetic circuit which can add 3 binary bits.
- A subtractor is used to subtract one binary bit from another.
- Full subtractor and half subtractor subtract one binary bit from another considering a previous borrow.
- Code converters are combinational logic circuits that convert one code to another code.
- Code converters are used in practical applications, e.g., in calculator's display screen and in LED panels display, 7-segment display is required.
- A binary comparator is used to compare the magnitudes of two binary numbers.
- A parallel adder adds two binary numbers in parallel from, (i.e., it accepts all inputs simultaneously and provides one output at a time.)
- A ripple carry is a parallel adder, in which carry-out of each full adder stage is taken as carry-in of next stage adder.
- Parallel adders are faster than serial adders.
- Look ahead carry adder has a minimum time delay to get a result.
- BCD adders are used in computers or calculators to produce an output in decimal from.

Practice Questions

1. A half adder can be constructed using:

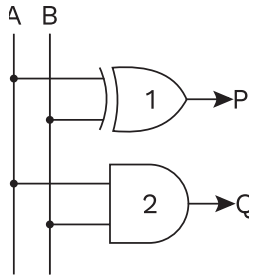
(A) One XOR and one OR gate with their outputs.

(B) One XOR and one OR gate with their outputs connected in series.

(C) One XOR gate and one AND gate.

(D) Two XNOR gates only.

2. A half-adder circuit is shown below having inputs AB = 11. The logic level of P and Q outputs will be:



(A) P = 0 and Q = 0

(B) P = 0 and Q = 1

(C) P = 1 and Q = 0

(D) P = 1 and Q = 1

3. Consider the following statements:

A half-adder

I. can work as a half subtractor using an inverter.

II. having outputs C = x · y and S = x ⊕ y for 2-inputs x and y.

III. having outputs C = x + y and S = x ⊕ y for 2-inputs x and y.

IV. is a type of combinational circuit.

Which of the above following statement(s) is/are correct?

(A) I, III and IV

(B) I, II and IV

(C) IV only

(D) II and III

4. A half adder circuit having two inputs A and B and 2-outputs, sum (S) and carry (C), the expression for sum and carry is-

(A) $S = \bar{A}B + A\bar{B};\ C = A \times B$

(B) $S = AB + \bar{A}B;\ C = A + B$

(C) $S = \bar{A}\bar{B} + AB;\ C = A + \bar{B}$

(D) $S = \bar{A} + A\bar{B};\ C = \bar{A} + B$

5. A 1-bit full adder takes 20 ns to generate a carry-out bit and 40 ns for the sum bit. The maximum rate of addition per second when four 1-bit full adders are cascaded is:

(A) 10^7

(B) 1.25×10^7

(C) 6.25×10^6

(D) 10^5

6. The logic function (f = AB + BC + CA), defines which one of the following:

I. The output of a 3-input XOR gate.

II. The output of a 3-input XNOR gate.

III. The sum of a full adder.

IV. The carry of a full adder.

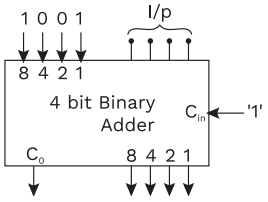
(A) I and II

(B) II and III

(C) III and IV

(D) Only IV

7. The below circuit is converting:



(A) Input number to Excess-3 number

(B) Input number 2's complement number

(C) Input number to 9's complement number

(D) None

8. Design a logic circuit with 4 inputs A, B, C and D that will produce an output '1' only when two adjacent input variables are 1's. A and D also have to be treated as adjacent.

(A) $F = AC + AD + BD + AB$

(B) $F = \bar{A}C + \bar{A}D + AC + BD$

(C) $F = AB + AD + BC + CD$

(D) $F = \bar{A}B + \bar{A}D + BC + CD$

Answer Keys

1. Solution: (C) **2. Solution: (B)** **3. Solution: (B)** **4. Solution: (A)**

5. Solution: (A) **6. Solution: (D)** **7. Solution: (D)** **8. Solution: (C)**

5 Combinational Circuits-II

Multiplexer

Multiplexer is used to convert parallel data to serial data. In multiplexer, if 2^n data are applied at an input port, then one of the input will appear at the output port with the help of 'n' select lines. That is why multiplexer is also known as data selector (many to one).

Block diagram:

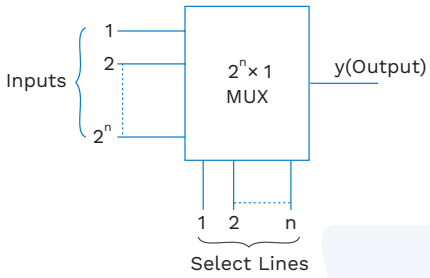


Fig. 5.1 Mux

Grey Matter Alert!

For implementation of basic gates, the required number of 2:1 MUX is 1.

2×1 MUX:

- The 2 × 1 multiplexer comprises 2-input bits, 1-control bit (select line) and 1-output bit.
- Here, we have considered I_0 and I_1 as input bits and y as output bit.
- Inputs of 2 × 1 MUX are I_0 and I_1, but only one of the input bit is transmitted to the output port.
- The control bit (select line) decides which one of the input data bit will transmit to the output port.
- Only 1 input will transmit at the output depending on the select line input.
  Inputs = 2, Output = 1
  Select line = 1

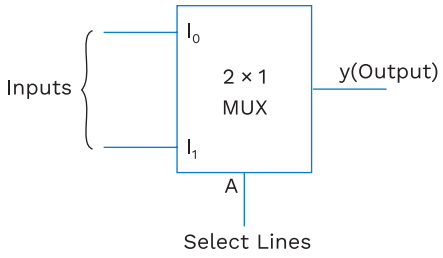


Fig. 5.2 2 x 1 Mux

4×1 MUX:

Previous Years' Questions

The logic realized by the circuit shown in figure is:

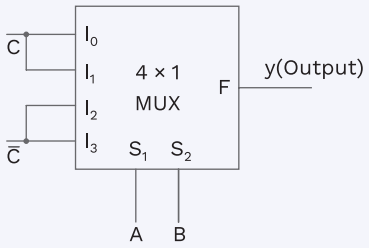


Fig. 5.3

(A) $F = A \odot C$ (B) $F = A \oplus C$
(C) $F = B \odot C$ (D) $F = B \oplus C$
Solution: (B)

- The 4 × 1 multiplexer comprises four-input bits, 2-control bits (select lines) and one-output bit.
- The 4-input bits are I_0, I_1, I_2 and I_3, but only one of the input bit will transmit to the output port.
- The control bits (select lines) decide which one of the input data bit should be transmitted to the output port.
  Inputs = 4
  Output = 1
  Select lines = 2

Block diagram:

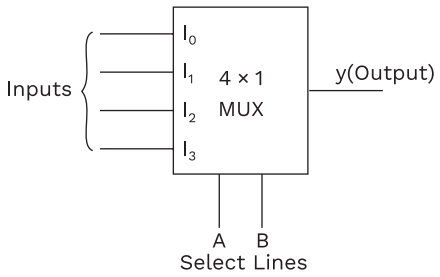


Fig. 5.4 4 x 1 Mux

Rack Your Brain

What is the minimum size of the multiplexer necessary if we have one MUX with one inverter that can implement any Boolean function with n variables?

8×1 MUX:

- Similarly for 8 × 1 MUX, input bits, control bits (select lines) and output bits are:
  Inputs = 8
  Output = 1
  Select lines = 3
  Block diagram:

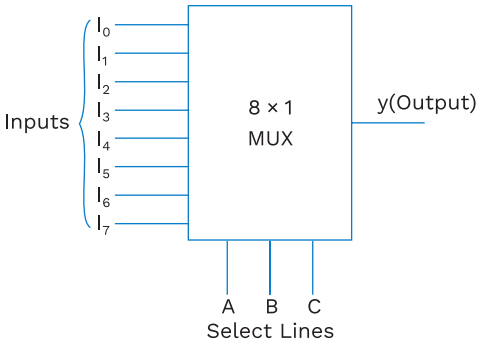


Fig. 5.5 B x 1 Mux

Example 1:

Design the given function by using 8 × 1 MUX.

Y(A, B, C) = ∑m (0, 2, 4, 5, 7)

Solution:

Variables = 3(A, B, C)

Select lines = 3

Block diagram:

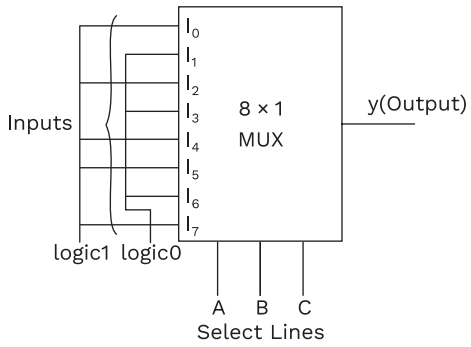


Fig. 5.6 8 x 1 Mux

Previous Years' Questions

In the TTL circuit in the figure, S_2, S_1 and S_0 are select lines and X_7 to X_0 are input lines. If S_0 and X_0 are LSB bits of select lines and input bits, respectively, then the output Y is:

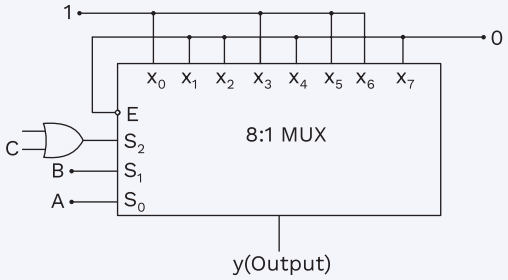


Fig. 5.7

(A) Indeterminate
(B) A XOR B
(C) $\overline{A \oplus B}$
(D) $\overline{C}(\overline{A \oplus B}) + C(A \oplus B)$

Solution: (B)

Example 2:

Implement the given function y(A, B) = ∑m (0, 2, 3) using 4 × 1 MUX.

Solution:

Variables = 2(A, B)

Select lines = 2

Block diagram:

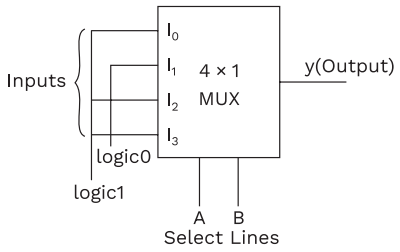


Fig. 5.8 4 x 1 Mux

Example 3:

Implement the given function

y(A, B, C, D) = ∑m(0, 2, 5, 8, 10, 12, 14, 15) using 16 × 1 MUX.

Solution:

Variable = 4(A, B, C, D)

Select lines = 4

Block diagram:

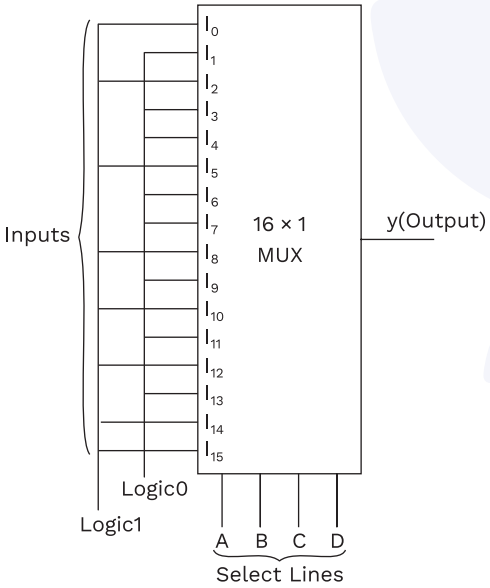


Fig. 5.9 16 x 1 Mux

Boolean Function Implementation by using MUX

For implementing the n-variable Boolean function using $2^n \times 1$ MUX, we require certain steps given as:

1. Express the given Boolean function in sum of product form.
2. Implement the Boolean function into truth table form.
3. For $2^n \times 1$ MUX, choose any one bit as input and remaining bits as select lines.
4. In two rows, list all the input minterms. Where a single variable is complemented in the first row, and all the minterms in uncomplemented form in the second row.
5. All of the function's minterms should be circled. Put '0' to the corresponding MUX input if two minterms in a column are not circled. Put '1' for the input if two minterms in a column are circled. If only one minterm is circled, the MUX input will be the complemented or uncomplemented front value.

Previous Years' Questions

A combinational circuit diagram is given below. I_0, I_1, I_2, and I_3 are inputs to the 4:1 multiplexer. R(MSB) and S are control bits.

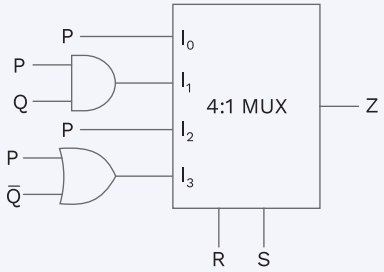


Fig. 5.10

The output Z(P, Q, R, S) can be represented by:

(A) $Z(P, Q, R, S) = PQ + P\bar{Q}S + \bar{Q}\bar{R}\bar{S}$

(B) $Z(P, Q, R, S) = P\bar{Q} + PQ\bar{R} + \bar{P}\bar{Q}\bar{S}$

(C) $Z(P, Q, R, S) = P\bar{Q}\bar{R} + \bar{P}QR + PQRS + \bar{Q}\bar{R}\bar{S}$

(D) $Z(P, Q, R, S) = PQ\bar{R} + PQR\bar{S} + P\bar{Q}\bar{R}S + \bar{Q}\bar{R}\bar{S}$

Solution: (A)

Example 4:

Implement the following function using 8 × 1 MUX, when select lines are A, B and C.

y(A, B, C, D) = ∑m(0, 2, 3, 5, 8, 10, 12, 15)

Solution:

Variables = 4(A, B, C, D)

Select lines = 3

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
$\bar{D}$ = 0	⓪	②	4	6	⑧	⑩	⑫	14
D = 1	1	③	⑤	7	9	11	13	⑮
	$\bar{D}$	1	D	0	$\bar{D}$	$\bar{D}$	$\bar{D}$	D

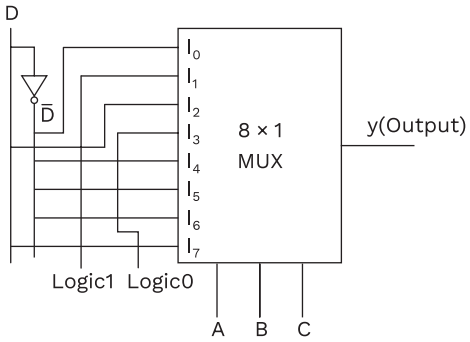


Fig. 5.11 8 x 1 Mux

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
$\bar{B}$ = 0	⓪	1	②	③	⑧	9	⑩	11
B = 1	4	⑤	6	7	⑫	13	14	⑮
	$\bar{B}$	B	$\bar{B}$	$\bar{B}$	1	0	$\bar{B}$	B

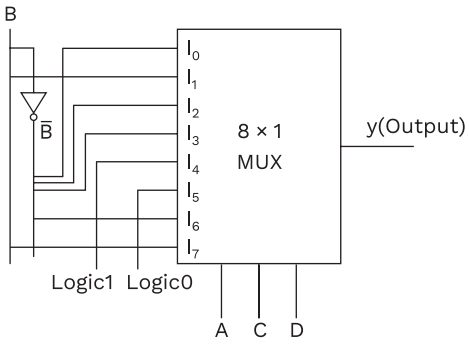


Fig. 5.13 8 x 1 Mux

Previous Years' Questions

In the given circuit, find the output of 4-1 multiplexer X(A, B, C):

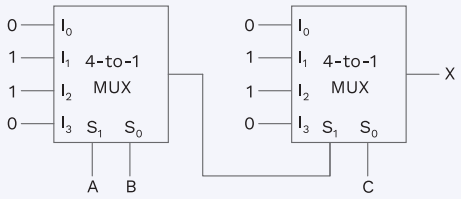


Fig. 5.12

(A) $X(A, B, C) = A\bar{B}\bar{C} + \bar{A}BC + \bar{A}\bar{B}C + ABC$

(B) $X(A, B, C) = \bar{A}BC + A\bar{B}C + AB\bar{C} + \bar{A}\bar{B}\bar{C}$

(C) $X(A, B, C) = AB + BC + AC$

(D) $X(A, B, C) = \bar{A}\bar{B} + \bar{B}\bar{C} + \bar{A}\bar{C}$

Solution: (A)

Example 5:
Implement the following function using 8 × 1 MUX, when A, C and D are select lines.

$y(A, B, C, D) = \sum m(0, 2, 3, 5, 8, 10, 12, 15)$

Solution:
Variables = 4(A, B, C, D)

Select lines = 3

Rack Your Brain

Implement the given Boolean function using 4 × 1 MUX.
$F(A, B, C) = \sum m(0, 1, 3, 5, 7)$

Example 6:
Implement the following function using 4 × 1 MUX, when select lines are A and B.

$y(A, B, C, D) = \sum m(0, 2, 3, 5, 8, 10, 12, 15)$

Solution:
Variables = 4(A, B, C, D)

Select lines = 2

	I_0	I_1	I_2	I_3
$\bar{C}\,\bar{D}$ = 00	⓪	4	⑧	⑫
$\bar{C}$ D = 01	1	⑤	9	13
C $\bar{D}$ = 10	②	6	⑩	14
C D = 11	③	7	11	⑮

$I_0 = \bar{C}\bar{D} + C\bar{D} + CD = \bar{C}\bar{D} + C(\bar{D} + D)$

$I_0 = \bar{C}\bar{D} + C = (C + \bar{C})(C + \bar{D}) = C + \bar{D}$

$I_1 = \bar{C}D$

$I_2 = \bar{C}\bar{D} + C\bar{D}$

$I_2 = \bar{D}$

$I_3 = \bar{C}\bar{D} + CD$

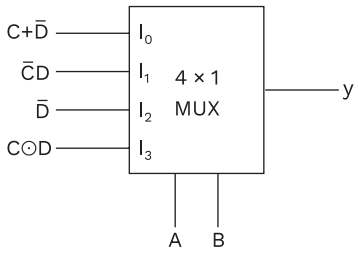


Fig. 5.14 4 x 1 Mux

Example 7:

Implement the following functions using 8 × 1 MUX.

1. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14) and select lines are A, B and C.
2. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14, 15) and select lines are A, C and D.
3. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14) and select lines are B, C and D.
4. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14) and select lines are A, B and D.

Solution:

1. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14) and select lines are A, B and C.
   Variables = 4(A, B, C, D)
   Select lines = 3

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
$\bar{D}$ = 0	0	2	4	(6)	8	10	(12)	(14)
D = 1	(1)	(3)	5	7	(9)	11	(13)	15
	D	D	0	$\bar{D}$	D	0	1	$\bar{D}$

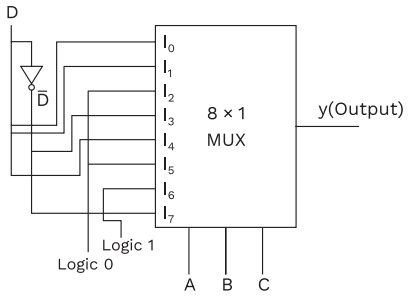


Fig. 5.15 8 x 1 Mux

2. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14, 15) and select lines are A, C and D.
   Variables = 4(A, B, C, D)
   Select lines = 3

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
$\bar{B}$ = 0	0	(1)	2	(3)	8	(9)	10	11
B = 1	4	5	(6)	7	(12)	(13)	(14)	(15)
	0	$\bar{B}$	B	$\bar{B}$	B	1	B	B

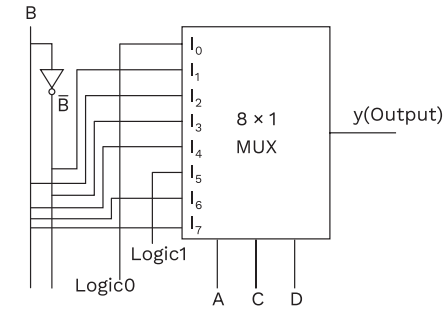


Fig. 5.16 8 x 1 Mux

3. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14) and select lines are B, C and D.
   Variables = 4(A, B, C, D)
   Select lines = 3

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
$\bar{A}$ = 0	0	(1)	2	(3)	4	5	(6)	7
A = 1	8	(9)	10	11	(12)	(13)	(14)	15
	0	1	0	$\bar{A}$	A	A	1	0

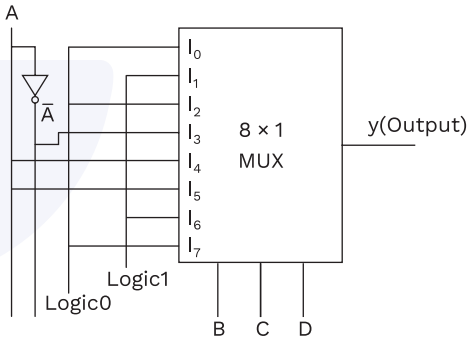


Fig. 5.17

Previous Years' Questions

A 4:1 multiplexer circuit is given below. The minterms of output F(A, B, C, D) is:

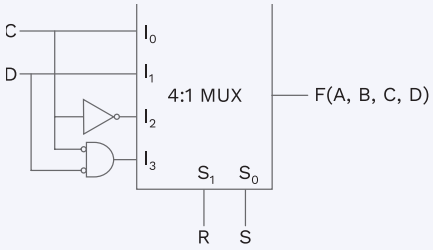


Fig. 5.18

(A) F(A, B, C, D) = ∑m(0, 1, 3, 5, 9, 10, 14)
(B) F(A, B, C, D) = ∑m(2, 3, 5, 7, 8, 12, 13)
(C) F (A, B, C, D) = ∑m(1, 2, 4, 5, 11, 14, 15)
(D) F (A, B, C, D) = ∑m(2, 3, 5, 7, 8, 9, 12)

Solution: (D)

4. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14) and select lines are A, B and D.
   Variables = 4(A, B, C, D)
   Select lines = 3

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
$\bar{C}$ = 0	0	①	4	5	8	⑨	⑫	⑬
C = 1	2	③	⑥	7	10	11	⑭	15
	0	1	C	0	0	$\bar{C}$	1	$\bar{C}$

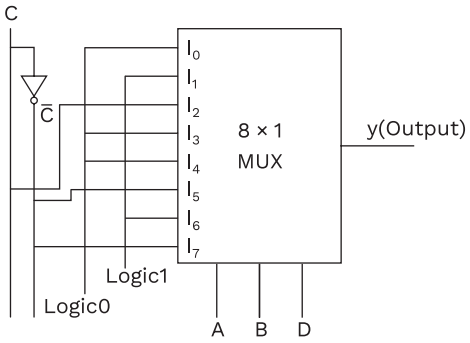


Fig. 5.19 8 x 1 Mux

Example 8:

Implement the following function using 4 × 1 MUX if two bits are considered as select lines.

1. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14) and select lines are C and D.
2. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14) and select lines are A and C.

Solution:

1. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14) and select lines are C and D.
   Variable = 4(A, B, C, D)
   Select lines = 2

	I_0	I_1	I_2	I_3
$\bar{A}\ \bar{B}$ = 00	0	①	2	③
$\bar{A}$ B = 01	4	5	⑥	7
A $\bar{B}$ = 10	8	⑨	10	11
A B = 11	⑫	⑬	⑭	15

$I_0 = AB$,

$I_1 = \bar{A}\bar{B} + A\bar{B} + AB$,

$I_1 = \bar{B}(\bar{A} + A) + AB$

$I_1 = AB + \bar{B}$

$I_1 = A + \bar{B}$

$I_2 = \bar{A}B + AB$

$I_2 = B(\bar{A} + A)$

$I_2 = B$

$I_3 = \bar{A}\bar{B}$

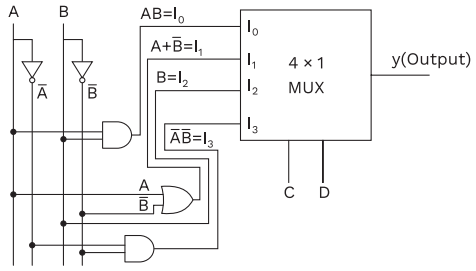


Fig. 5.20 Implementation using 4 x 1 Mux

2. y(A, B, C, D) = ∑m(1, 3, 6, 9, 12, 13, 14) and select lines are A and C.
   Variable = 4(A, B, C, D)
   Select lines = 2

	I_0	I_1	I_2	I_3
$\bar{B}\ \bar{D}$ = 00	0	2	8	10
$\bar{B}$ D = 01	①	③	⑨	11
B $\bar{D}$ = 10	4	⑥	⑫	⑭
B D = 11	5	7	⑬	15

$I_0 = \bar{B}D$

$I_1 = \bar{B}D + B\bar{D}$

$I_2 = \bar{B}D + B\bar{D} + BD$

$I_2 = B\bar{D} + D$

$I_3 = B\bar{D}$

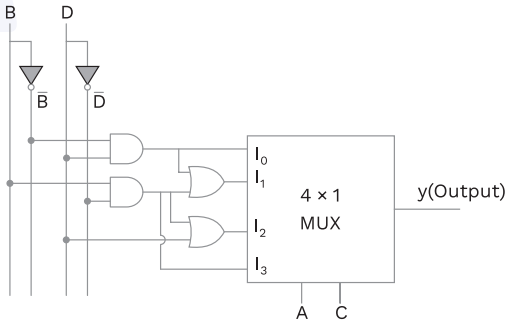


Fig. 5.21 Implementation using 4 x 1 Mux

Example 9:

Using 8 × 1 MUX, implement the following function.When D is an input line. y(A, B, C, D) = πM(0, 2, 5, 6, 10, 13, 14, 15)

Solution:

Given,

y(A, B, C, D) = πM(0, 2, 5, 6, 10, 13, 14, 15)

or, y(A, B, C, D) = ∑m(1, 3, 4, 7, 8, 9, 11, 12)

Using 8 × 1 MUX

Select lines = 3

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
$\bar{D}$ = 0	0	2	④	6	⑧	10	⑫	14
D = 1	①	③	5	⑦	⑨	⑪	13	15
	D	D	$\bar{D}$	D	1	D	$\bar{D}$	0

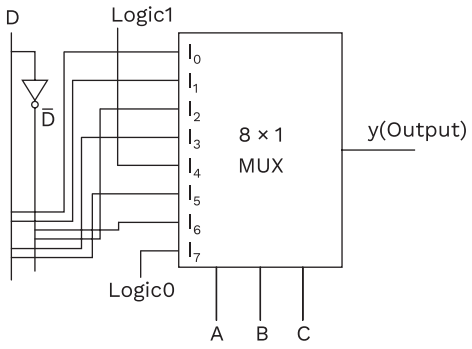


Fig. 5.22 8 x 1 Mux

Previous Years' Questions

The logic function is implemented by using 4:1 multiplexer, then the output of the multiplexer is (ground implies a logic '0'):

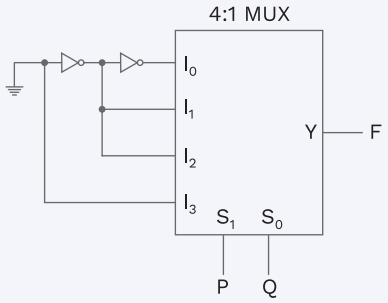


Fig. 5.23

(A) F(P, Q) = P AND Q
(B) F(P, Q) = P OR Q
(C) F(P, Q) = P XNOR Q
(D) F(P, Q) = P XOR Q

Solution: (D)

Example 10:

Implement the following function using 4 × 1 MUX.

y(A, B, C) = πM(0, 2, 3, 4, 7). If C is a input line.

Solution:

Given, y(A, B, C) = πM(0, 2, 3, 4, 7)

or, y(A, B, C) = ∑m(1, 5, 6)

Using 4 × 1 MUX,

Select lines = 2

	I_0	I_1	I_2	I_3
$\bar{C}$ = 0	0	2	4	⑥
C = 1	①	3	⑤	7
	C	0	C	$\bar{C}$

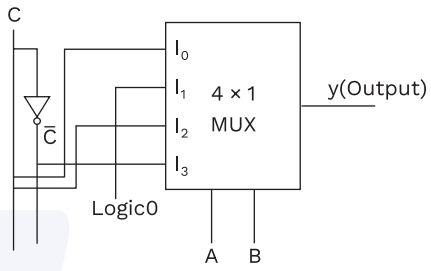


Fig. 5.24 4 x 1 Mux

Example 11:

Using 8 × 1 multiplexer, implement a logical circuit for a full adder.

Solution:

Truth table:

Inputs			Outputs	
A	B	C_{in}	S	C_0
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 5.1

$S(A, B, C_{in}) = \sum m(1, 2, 4, 7)$ and
$(A, B, C_{in}) = \sum m(3, 5, 6, 7)$

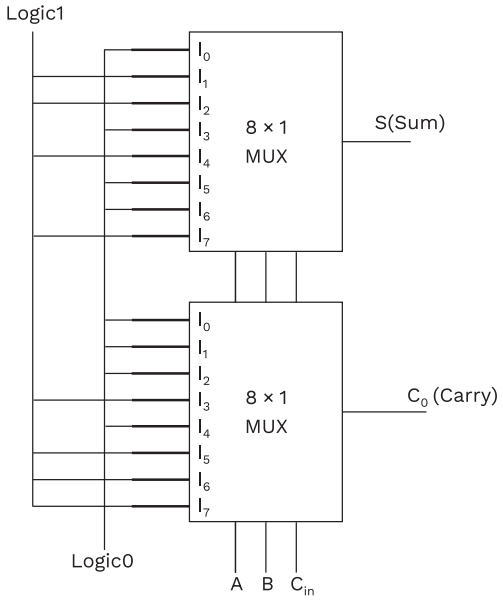


Fig. 5.25 Implementation using 8 x 1 Mux

Example 12:

Implement full subtractor using MUX.

Solution:

The truth table of full subtractor is shown below:

Inputs			Outputs	
A	B	C	D	B_0
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Table 5.2

$D(A, B, C_{in}) = \sum m(1, 2, 4, 7)$ and, $B_0(A, B, C_{in}) = \sum m(1, 2, 3, 7)$

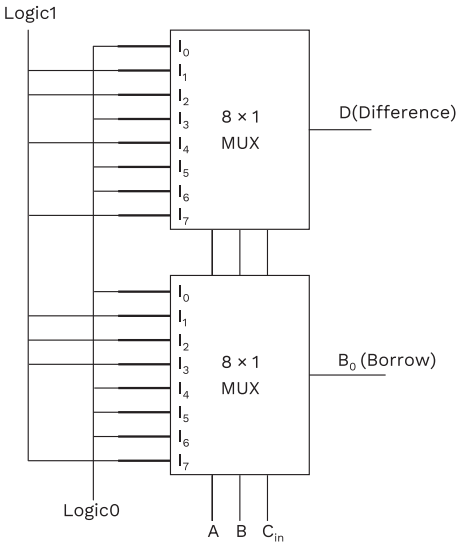


Fig. 5.26 Implementation using 8 x 1 Mux

Example 13:

Implement the logical circuit of half adder using 4 × 1 multiplexer.

Solution:

The truth table of half adder is shown below:

A	B	S	C_0
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 5.3

Sum, $S = \sum m(1, 2)$

Carry, $C_0 = \sum m(3)$

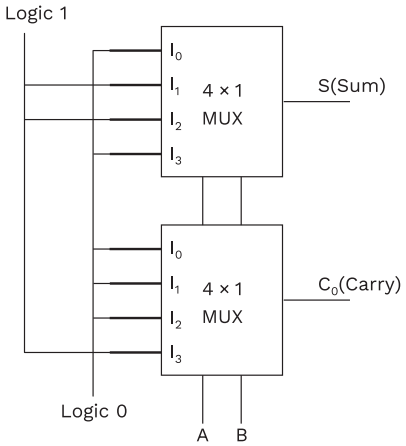


Fig. 5.27 Implementation using 4 x 1 Mux

Previous Years' Questions

The Boolean function f(A, B, C) implemented in the given figure using 2 × 1 multiplexers is:

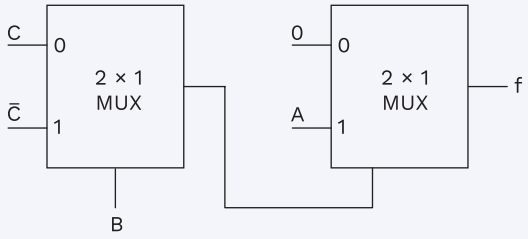


Fig. 5.27

(A) $F(A,B,C) = A\bar{B}C + AB\bar{C}$

(B) $F(A,B,C) = ABC + A\bar{B}\bar{C}$

(C) $F(A,B,C) = \bar{A}BC + \bar{A}\bar{B}\bar{C}$

(D) $F(A,B,C) = \bar{A}\bar{B}C + \bar{A}B\bar{C}$

Solution: (A)

Example 14:

Implement half subtractor using 4 × 1 multiplexer.

Solution:

The truth table of half subtractor is shown below:

A	B	Borrow (B_0)	Difference (D)
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

Table 5.4

Difference, D(A, B) = ∑m(1, 2)

Borrow, B_0(A, B) = ∑m(1)

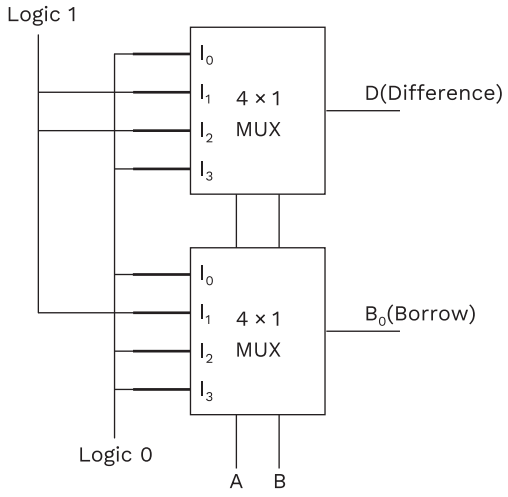


Fig. 5.28 Implementation of Half Subtractor Using 4 x 1 Mux

Example 15:

Implement full adder using 4 × 1 MUX, when two input bits are select lines.

Solution:

The truth table of full adder is shown below:

Inputs			Outputs	
A	B	C_{in}	S	C_0
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 5.5

Sum (S) (A, B, C_{in}) = ∑m(1, 2, 4, 7)

Carry (C_0) (A, B, C_{in}) = ∑m(3, 5, 6, 7)

Select lines = 2

For Sum (S):

	I_0	I_1	I_2	I_3
$\bar{C}_{in} = 0$	0	②	④	6
$C_{in} = 1$	①	3	5	⑦
	C_{in}	$\bar{C}_{in}$	$\bar{C}_{in}$	C_{in}

For Carry (C_0):

	I_0	I_1	I_2	I_3
$\bar{C}_{in} = 0$	0	2	4	⑥
$C_{in} = 1$	1	③	⑤	⑦
	0	C_{in}	C_{in}	1

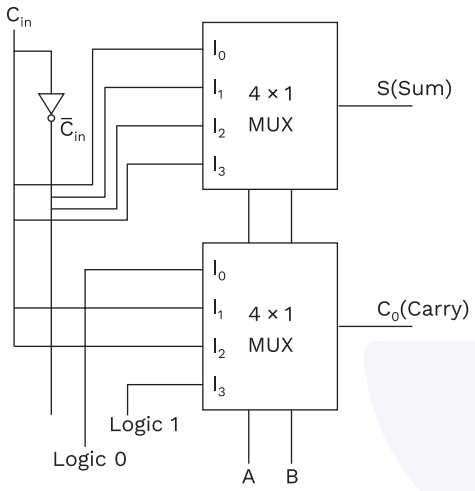


Fig. 5.29 Implementation of Full Adder Using 4 x 1 Mux

Example 16:

Implement AND gate & OR gate using X-type gate shown below whose output is $\bar{P}Q$ and inputs are P & Q.

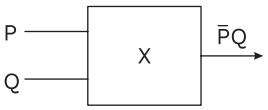


Fig. 5.30 Implementation of Logic Gate

Solution:

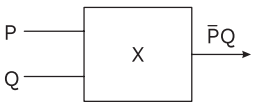


Fig. 5.31 Implementation of Logic Gate Using Mux

Implementation of AND gate:

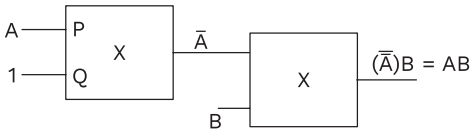


Fig. 5.32 Implementation of AND Gate Using Mux

Implementation of OR gate:

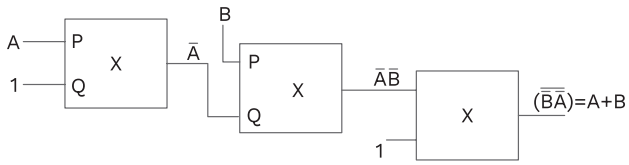


Fig. 5.33 Implementation of OR Gate Using Mux

Example 17:

Implement full subtractor using 4 × 1 MUX.

Solution:

The truth table of full subtractor is shown below:

Inputs			Outputs	
A	B	B_{in}	D	B_0
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Table 5.6

Select B_{in} at input port,
Difference (A, B, B_{in}) = $\sum$m(1, 2, 4, 7)
Borrow (A, B, B_{in}) = $\sum$m(1, 2, 3, 7)
Select lines = 2
Difference (D):

	I_0	I_1	I_2	I_3
$\bar{B}_{in} = 0$	0	②	④	6
$B_{in} = 1$	①	3	5	⑦
	B_{in}	$\bar{B}_{in}$	$\bar{B}_{in}$	B_{in}

Borrow (B_0):

	I_0	I_1	I_2	I_3
$\bar{B}_{in} = 0$	0	②	4	6
$B_{in} = 1$	①	③	5	⑦
	B_{in}	1	0	B_{in}

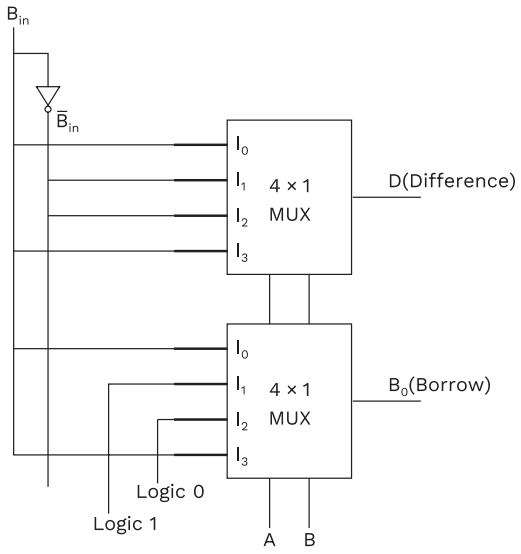


Fig. 5.34 Implementation of Full subtractor using Mux

Example 18:

Implement half adder using 2 × 1 MUX.

Solution:

Write the truth table of half adder:

A	B	Carry (C_0)	Sum (S)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Table 5.7

Select B at input port,
Sum (A, B) = $\sum m(1, 2)$
Carry (A, B) = $\sum m(3)$
Select line = 1
For Sum (S):

	I_0	I_1
$\bar{B} = 0$	0	(2)
$B = 1$	(1)	3
	B	$\bar{B}$

$S = \bar{A}B + A\bar{B}$

$S = A \oplus B$

For Carry (C_0):

	I_0	I_1
$\bar{B} = 0$	0	2
$B = 1$	1	(3)
	0	B

$C_0 = AB$

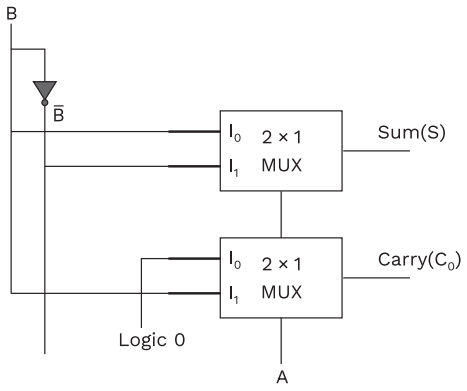


Fig. 5.35 Implementation of Half Adder Using Mux

Previous Years' Questions

In the given combinational circuit, W and Y are MSBs of the select bits for both the multiplexers. The output is given by:

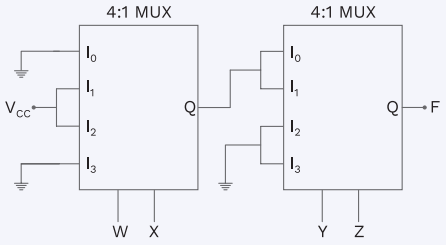


Fig. 5.36

(A) $F(W, X, Y, Z) = W\bar{X} + \bar{W}X + \bar{Y}\bar{Z}$

(B) $F(W, X, Y, Z) = W\bar{X} + \bar{W}X + \bar{Y}Z$

(C) $F(W, X, Y, Z) = W\bar{X}\bar{Y} + \bar{W}X\bar{Y}$

(D) $F(W, X, Y, Z) = (\bar{W} + \bar{X})\bar{Y}\bar{Z}$

Solution: (C)

Example 19:

Using an 2 × 1 multiplexer, implement a logical circuit for a half subtractor.

Solution:

The truth table of half subtractor is shown below:

A	B	Borrow (B_0)	Difference (D)
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

Table 5.8

Select line = 1

D (A, B) = $\sum m(1, 2)$,

B_0 (A, B) = $\sum m(1)$

Difference (D):

	I_0	I_1
$\bar{B}$ = 0	0	(2)
B = 1	(1)	3
	B	$\bar{B}$

$D = \bar{A}B + A\bar{B}$

$D = A \oplus B$

Borrow (B_0):

	I_0	I_1
$\bar{B}$ = 0	0	2
B = 1	(1)	3
	B	0

$B_0 = \bar{A}B$

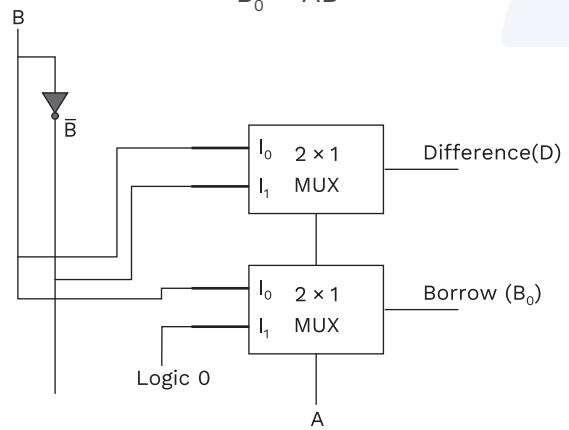


Fig. 5.37 Implementation of Half Subtractor Using Mux

Example 20:

Design three-bit binary to grey code converter using 8 × 1 MUX.

Solution:

The truth table of three-bit binary to grey code:

Binary			Grey code		
B_3	B_2	B_1	G_3	G_2	G_1
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	1	0	0

Table 5.9

Select lines = 3

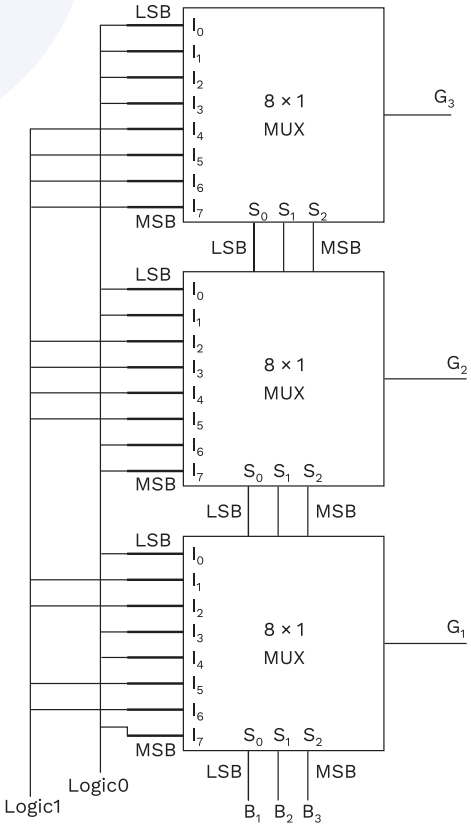


Fig. 5.38 Implementation of Binary to Gray Code using Mux

Example 21:

Implement 16 × 1 MUX using 4 × 1 MUX.

Solution:

16 × 1 MUX	4 × 1 MUX
Inputs = 16	Inputs = 4
Output = 1	Output = 1
Select lines = 4	Select lines = 2

Table 5.10

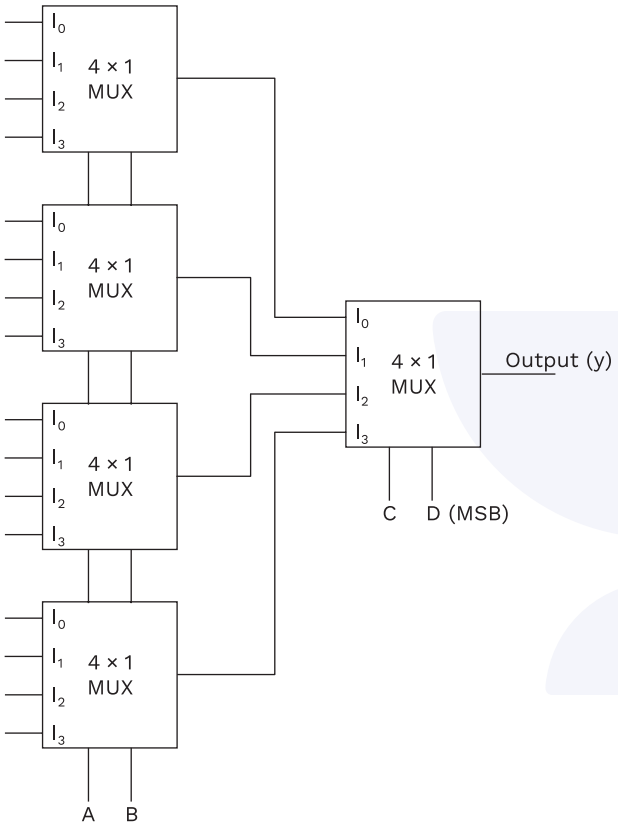


Fig. 5.39

Solution:

8 × 1 MUX	2 × 1 MUX
Inputs = 8	Inputs = 2
Output = 1	Output = 1
Select lines = 3	Select lines = 1

Table 5.11

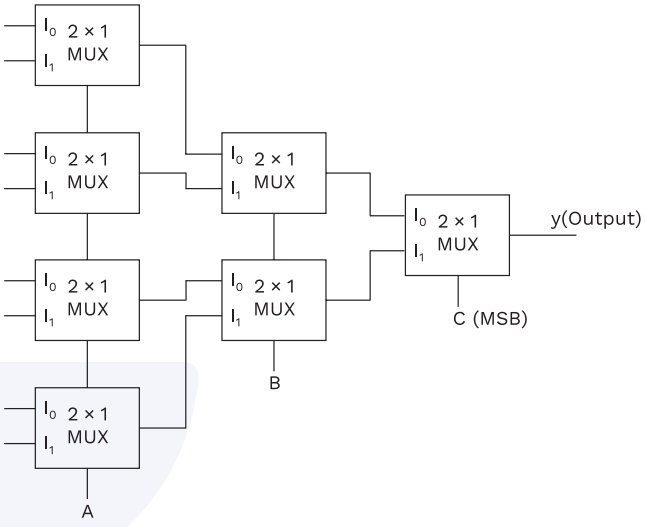


Fig. 5.40 Implementation of 8 x 1 Mux using 2 x 1 Mux

Demultiplexer (DEMUX)

Demultiplexer has one input, n select lines and 2^n output lines. It converts serial data into parallel data. It works as a data distributor (one to many).

Block diagram:

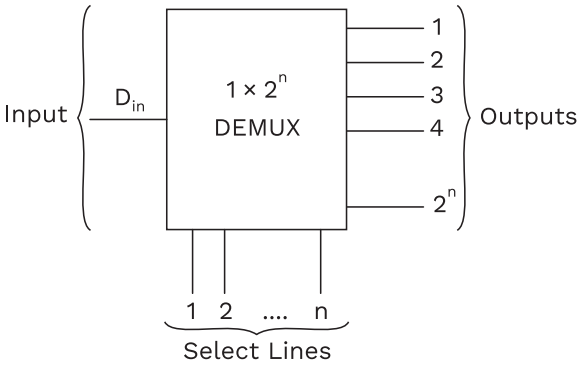


Fig. 5.41 DEMUX

1 × 4 demultiplexer:

- 1 × 4 demultiplexer requires 1 input bit (input line), 2 control bits (select lines) and 4 output bits (output lines).

Previous Years' Questions

Find the minimum number of 2 × 1 MUX's required to realize a 4 × 1 MUX:

(A) 1 (B) 2
(C) 3 (D) 4

Solution: (C)

Example 22:

Implement 8 × 1 MUX using 2 × 1 MUX.

- For a particular combination of select lines, the input can go to any one of the four outputs at a given time.

Block diagram of 1 × 4 de-MUX:

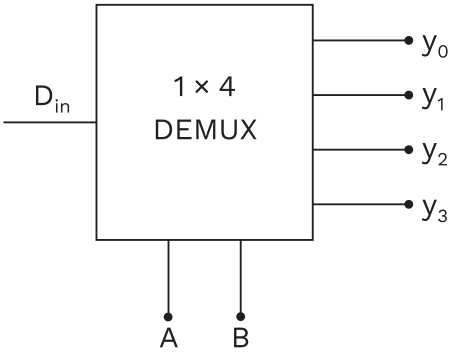


Fig. 5.42 1 X 4 DEMUX

The truth table:

A	B	y_3	y_2	y_1	y_0
0	0	0	0	0	D_{in}
0	1	0	0	D_{in}	0
1	0	0	D_{in}	0	0
1	1	D_{in}	0	0	0

Table 5.12

So, $y_0 = \bar{A}\bar{B}D_{in}$

$y_1 = \bar{A}BD_{in}$

$y_2 = A\bar{B}D_{in}$

$y_3 = ABD_{in}$

Circuit diagram:

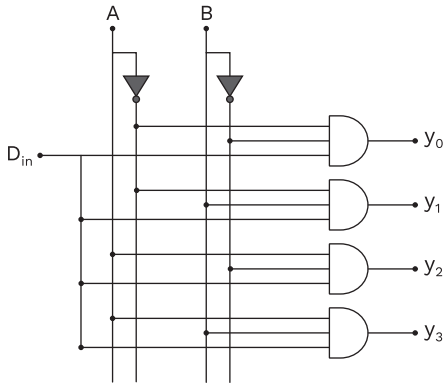


Fig. 5.43 Implementation of 1 X 4 DEMUX using Logic Gate

- **1 × 8 de-multiplexer:**

It consists:

Input bits = 1

Control bits = 3

Output bits = 8

Block diagram:

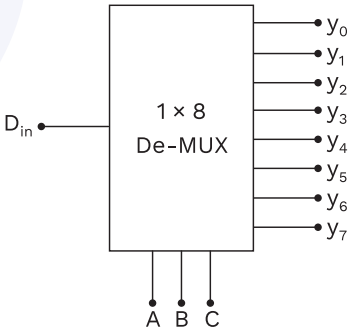


Fig. 5.44 1 X 8 DEMUX

The truth table:

A	B	C	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0
0	0	0	0	0	0	0	0	0	0	D_{in}
0	0	1	0	0	0	0	0	0	D_{in}	0
0	1	0	0	0	0	0	0	D_{in}	0	0
0	1	1	0	0	0	0	D_{in}	0	0	0
1	0	0	0	0	0	D_{in}	0	0	0	0
1	0	1	0	0	D_{in}	0	0	0	0	0
1	1	0	0	D_{in}	0	0	0	0	0	0
1	1	1	D_{in}	0	0	0	0	0	0	0

Table 5.13

So, $y_0 = \bar{A}\bar{B}\bar{C}\, D_{in}$

$y_1 = \bar{A}\bar{B}C\, D_{in}$

$y_2 = \bar{A}B\bar{C}\, D_{in}$

$y_3 = \bar{A}BC\, D_{in}$

$y_4 = A\bar{B}\bar{C}\, D_{in}$

$y_5 = A\bar{B}C\, D_{in}$

$y_6 = AB\bar{C}\, D_{in}$

$y_6 = ABC\, D_{in}$

Rack Your Brain

What is the number of gates required for a carry to propagate from input to output in an n-bit binary adder?

Circuit diagram:

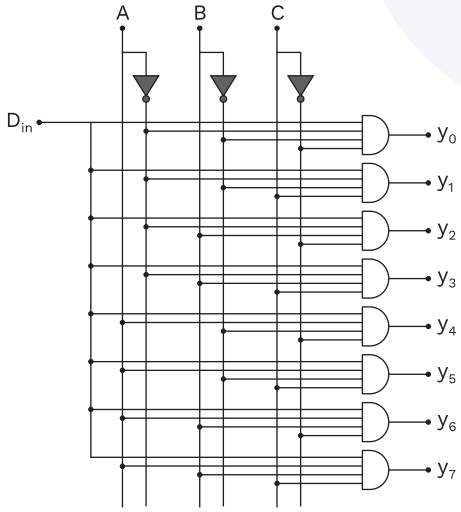


Fig. 5.45 Implementation of 1 X 8 DEMUX Using Logic Gate

Example 23:

Implement the following function using DEMUX.

1. $F_1(A, B, C) = \sum m(0, 1, 3, 7)$
2. $F_2(A, B, C) = \sum m(1, 2, 4, 6, 7)$

Solution:

Given,

1. $F_1(A, B, C) = \sum m(0, 1, 3, 7)$
2. $F_2(A, B, C) = \sum m(1, 2, 4, 6, 7)$

Using 1 × 8 DEMUX:

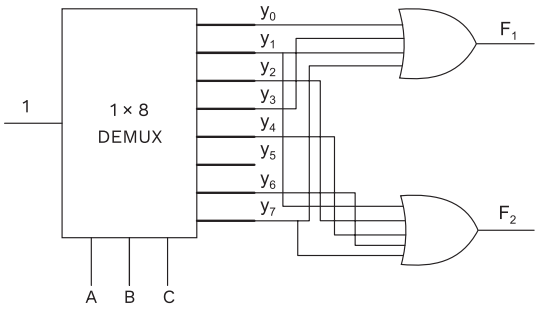


Fig. 5.46 Implementation of 1 X 8 DEMUX

Example 24:

Implement full adder using DEMUX.

Solution:

The truth table of full adder:

Inputs			Outputs	
A	B	C_{in}	S	C_0
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 5.14

$S(A, B, C_{in}) = \sum m(1, 2, 4, 7)$

and, $C_0(A, B, C_{in}) = \sum m(3, 5, 6, 7)$

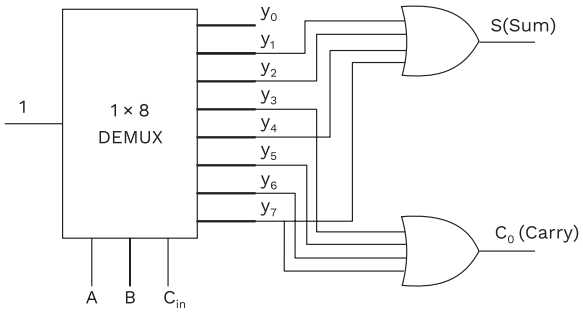


Fig. 5.47 Implementation of Full Adder using DEMUX

Example 25:

Implement the half adder using DEMUX.

Solution:

The truth table of half adder:

A	B	Carry (C_0)	Sum (S)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Table 5.15

$S(A, B) = \sum m(1, 2)$

and, $C_0(A, B) = \sum m(3)$

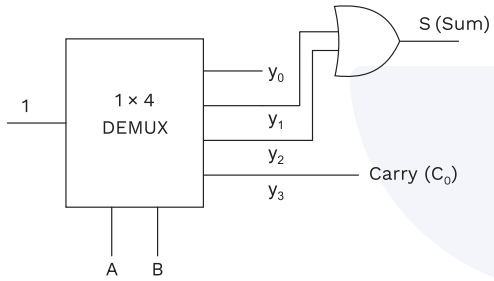


Fig. 5.48 Implementation of Half Adder Using DEMUX

Grey Matter Alert!

2 × 4 line decoder can be used in place of 1 × 4 DEMUX and vice-versa.

Example 26:

Using DEMUX, implement a half subtractor and a full subtractor.

Solution:

Half subtractor

The truth table of half subtractor:

A	B	Borrow (B_0)	Difference (D)
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

Table 5.16

$D(A, B) = \sum m(1, 2)$ and, $B_0(A, B) = \sum m(1)$

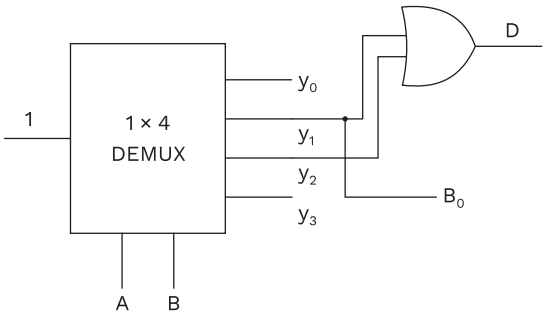


Fig. 5.49 Implementation of Half Subtractor Using DEMUX

Full subtractor

Truth table of full subtractor:

Inputs			Outputs	
A	B	C	D	B_0
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Table 5.17

Difference, $D(A, B, D_{in}) = \sum m(1, 2, 4, 7)$

and Borrow, $B_0(A, B, D_{in}) = \sum m(1, 2, 3, 7)$

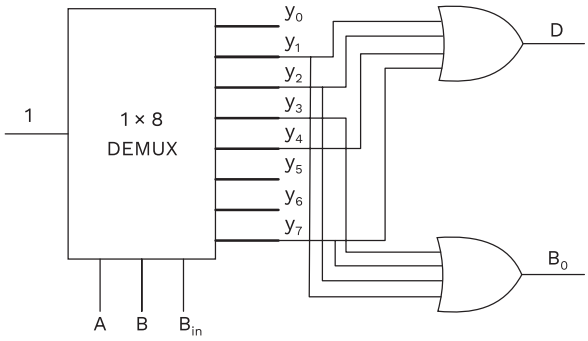


Fig. 5.50 Implementation of Full Subtractor Using DEMUX

Previous Years' Questions

The functionality implemented by the circuit below is:

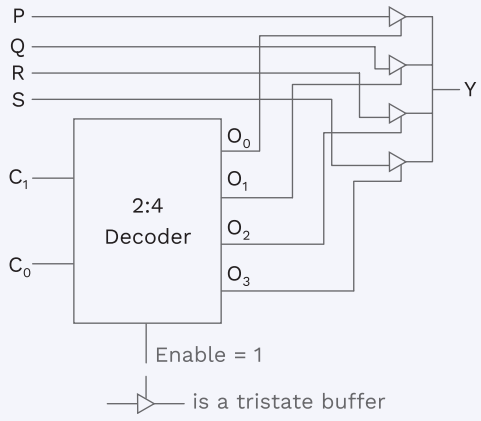


Fig. 5.51

(A) 2 × 1 MUX
(B) 4 × 1 MUX
(C) 7 × 1 MUX
(D) 6 × 1 MUX

Solution: (B)

Example 27:

Implement the following functions using DEMUX and OR gate.

$F_1(A, B) = \pi M(1)$ and $F_2(A, B) = \pi M(1, 3)$

Solution:

Given,

$F_1(A, B) = \pi M(1)$

or $F_1(A, B) = \sum m(0, 2, 3)$

$F_2(A, B) = \pi M(1, 3)$

or $F_2(A, B) = \sum m(0, 2)$

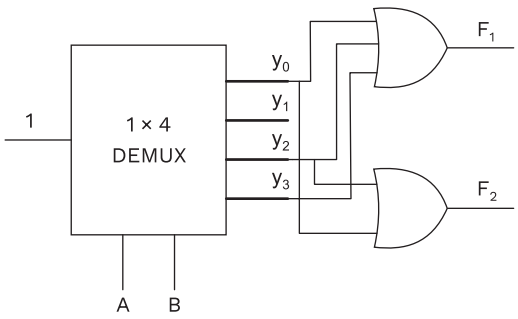


Fig. 5.52 Implementation of Function using DEMUX

Example 28:

Implement 1 × 16 DEMUX using 1 × 4 DEMUX.

Solution:

1 × 16 DEMUX	1 × 4 DEMUX
Input = 1	Input = 1
Outputs = 16	Outputs = 4
Input = 1	Input = 1
Outputs = 16	Outputs = 4
Select lines = 4	Select lines = 2

Table 5.18

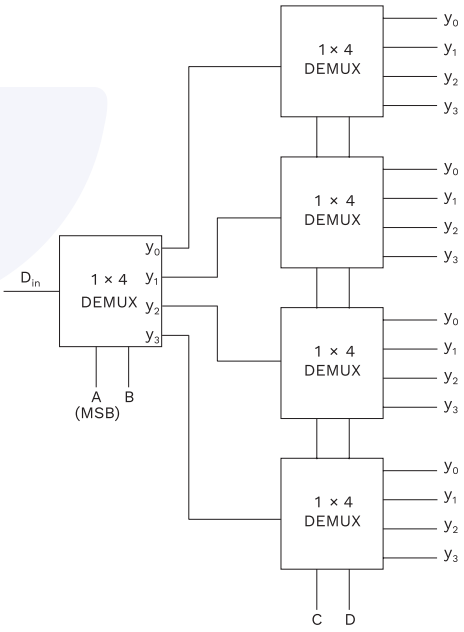


Fig. 5.53 Implementation of 1 x 16 DEMUX Using 1 X 4 DEMUX

Applications of multiplexer:

- Communication system: It is used to improve the communication system's efficiency.
- Telephone network: For transmission of multiple audio signals through the single line channel, multiplexers are used in telephone network.
- Waveform generation: Multiplexers are used for waveform generation of two or more input waveforms.

- Parallel-to-serial conversion: Multiplexer can be used to convert the parallel data input into the serial output.
- Data acquisition system: Multiplexer is the main component of data acquisition system.

Previous Years' Questions

If the input X_3, X_2, X_1, X_0 to the ROM in the figure are 8 4 2 1 BCD numbers, then the outputs Y_3, Y_2, Y_1, Y_0 of the given logic circuit are:

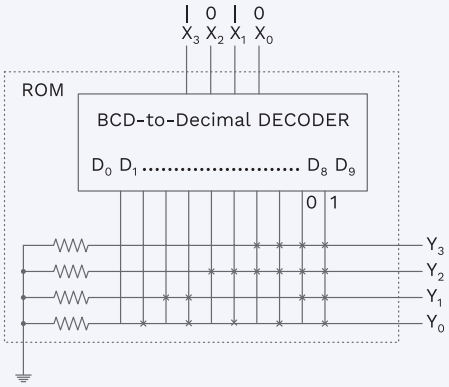


Fig. 5.54

(A) Excess-3 code numbers
(B) 2 4 2 1 BCD numbers
(C) Grey code numbers
(D) None of the above

Solution: (B)

Decoder

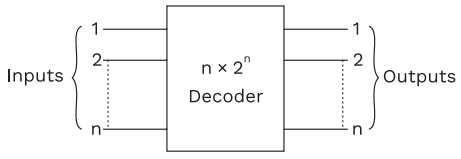


Fig. 5.55 Decoder

A decoder is a combinational logic circuit that translates an n-bit binary input code to 2^n output lines, with only one output line active for each of the possible input combinations.

2 × 4 decoder:

Inputs = 2

Outputs = 4

Block diagram:

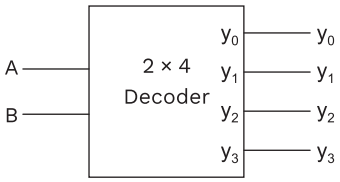


Fig. 5.56 2 X 4 Decoder

Rack Your Brain

To implement a 6 × 64 line decoder, how many numbers of 3 × 8 line decoders with an enable input is required without using any other logic gates?

The truth table:

Inputs		Output
A	B	y
0	0	y_0
0	1	y_1
1	0	y_2
1	1	y_3

Table 5.19

Expressions:

$y_0 = \bar{A}\bar{B}$, $y_1 = \bar{A}B$, $y_2 = A\bar{B}$, $y_3 = AB$

Logic diagram:

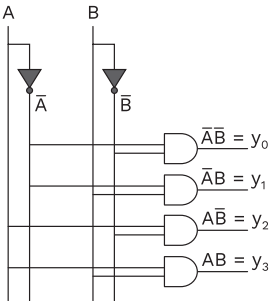


Fig. 5.57 Implementation of DEMUX using Logic Gate

3 × 8 decoder:

It consists:

Inputs = 3

Outputs = 8

Block diagram:

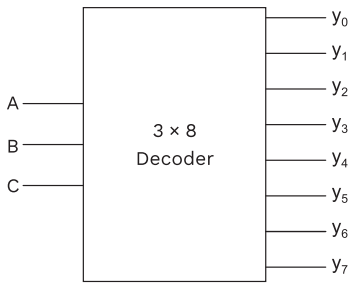


Fig. 5.58 3 X 8 Decoder

Grey Matter Alert!

Decoder contains AND-gates or NAND-gates.

The truth table:

Inputs			Output
A	B	C	y
0	0	0	y_0
0	0	1	y_1
0	1	0	y_2
0	1	1	y_3
1	0	0	y_4
1	0	1	y_5
1	1	0	y_6
1	1	1	y_7

Table 5.20

Expression:

$$y_0 = \bar{A}\bar{B}\bar{C}$$
$$y_1 = \bar{A}\bar{B}C$$
$$y_2 = \bar{A}B\bar{C}$$
$$y_3 = \bar{A}BC$$
$$y_4 = A\bar{B}\bar{C}$$
$$y_5 = A\bar{B}C$$
$$y_6 = AB\bar{C}$$
$$y_7 = ABC$$

Logic diagram:

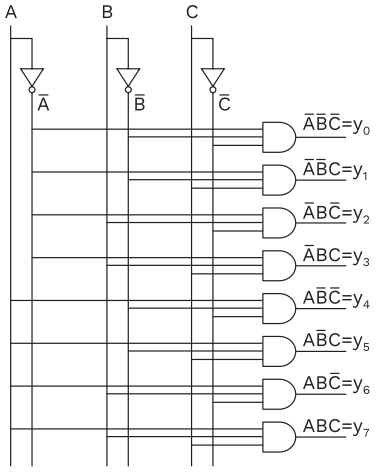


Fig. 5.59 Implementation of Decoder using Gate

Applications of decoder:

1. It is used for addressing purposes.
2. It is used as a code converter.
3. It is used for data distribution.

Example 29:

Implement full adder using 3 × 8 decoder.

Solution:

The truth table of full adder:

Inputs			Outputs	
A	B	C_{in}	Carry (C_0)	Sum (S)
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Table 5.21

Sum(A, B, C) = $\sum$m(1, 2, 4, 7)
Carry(A, B, C) = $\sum$m(3, 5, 6, 7)
Using 3 × 8 decoder,

Block diagram:

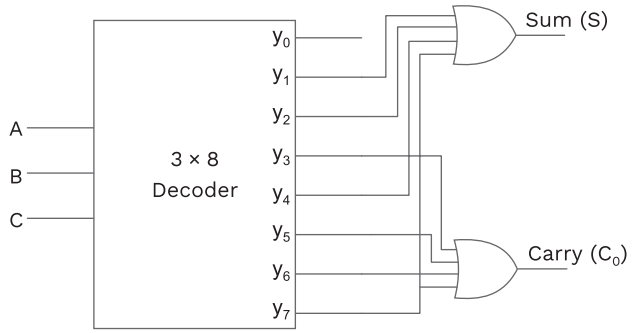


Fig. 5.60 3 X 8 Decoder

Example 30:

Implement F(a, b, c) = ∑m(0, 4, 5, 6) using 3–8 decoder and AND gate.

Solution:

F = ∑m(0, 4, 5, 6) = πM(1, 2, 3, 7)

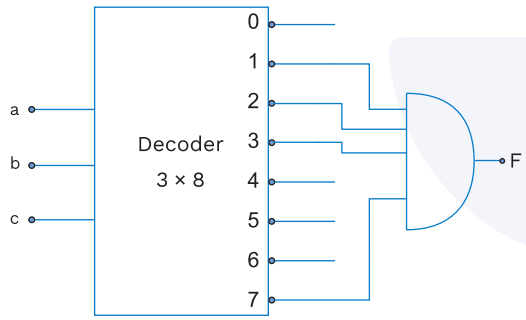


Fig. 5.61 Implementation using 3 X 8 Decoder and AND Gate

Output F will become zero only when at least one of the line is activated, else it will remain logic '1'.

Example 31:

Implement $F(a,b,c) = \bar{b} + c$ using 3–8 decoder and NOR gate.

Solution:

Given, $F = \bar{b} + c$

a \ bc	$\bar{b}\bar{c}$	$\bar{b}c$	bc	$b\bar{c}$
a	1 (0)	1 (1)	1 (3)	0 (2)
$\bar{a}$	1 (4)	1 (5)	1 (7)	0 (6)

Fig. 5.62 K-map

F = πM (2, 6)

F= ∑m (0, 1, 3, 4, 5, 7)

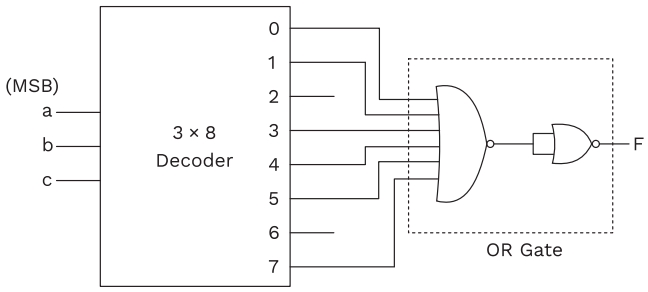


Fig. 5.63 Implementation using 3 X 8 Decoder and Gate

Example 32:

Implement half adder, half subtractor and full subtractor using decoder.

Solution:

Half adder

The truth table of half adder:

Inputs		Outputs	
A	B	Carry (C_0)	Sum (S)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Table. 5.22

S(A, B) = ∑m(1, 2) and, C_0(A, B) = ∑m(3)

Using 2 × 4 decoder,

Block diagram:

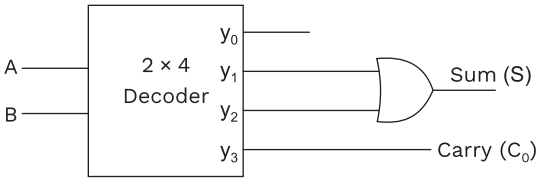


Fig. 5.64 Implementation of Half Adder using 2 X 4 Decoder

Half subtractor

The truth table of half subtractor:

Inputs		Outputs	
A	B	Borrow (B_0)	Difference (D)
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

Table 5.23

$D(A, B) = \sum m(1, 2)$ and, $B_0(A, B) = \sum m(1)$

Block diagram:

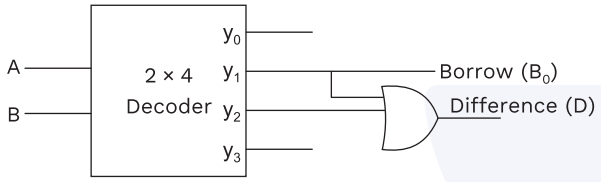


Fig. 5.65 Implementation of Half Subtractor using 2 X 4 Decoder

Full subtractor

The truth table of full subtractor:

Inputs			Outputs	
A	B	C	D	B_0
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Table 5.24

$D(A, B, C) = \sum m(1, 2, 4, 7)$ and, $B_0(A, B, C) = \sum m(1, 2, 3, 7)$

Circuit diagram:

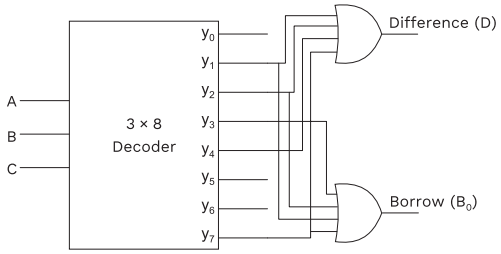


Fig. 5.66 Implementation of Full Subtractor using 3 X 8 Decoder

Encoders

- The encoder's operation is just opposite of the decoder.
- It has 2n input lines, and n output lines.
- An encoder accepts only one active level out of 2^n input lines and produces n output codes.
- Its input represents digit such as decimal, octal or hexadecimal digits and converts into an 'n' coded output lines.

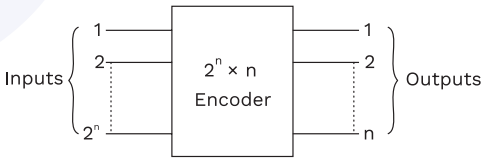


Fig. 5.67 Encoder

- Encoder converts codes such as,
  - Octal to binary (8 × 3 lines)
  - Decimal to BCD (10 × 4 lines)
  - Hexadecimal to binary (16 × 4 lines)

1. Octal to binary encoder:

Input lines = 8

Output lines = 3

Block diagram:

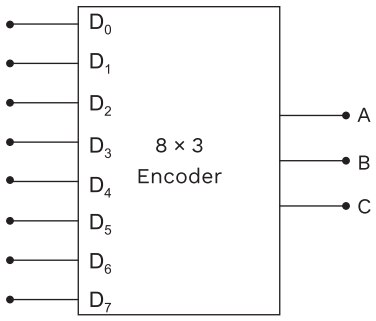


Fig. 5.68 Octal to Binary Enoder

The truth table:

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A	B	C
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Table 5.25

Output expressions

$A = D_4 + D_5 + D_6 + D_7$

$B = D_2 + D_3 + D_6 + D_7$

$C = D_1 + D_3 + D_5 + D_7$

Logic diagram of 8 × 3 encoder:

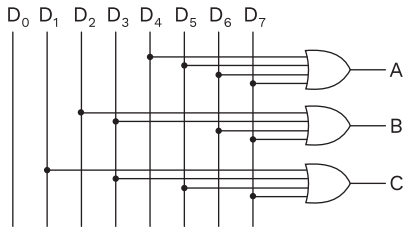


Fig. 5.69 Implementation of 8 X 3 Encoder using Logic Gate

2. Decimal to BCD encoder:

Input lines = 10

Output lines = 4

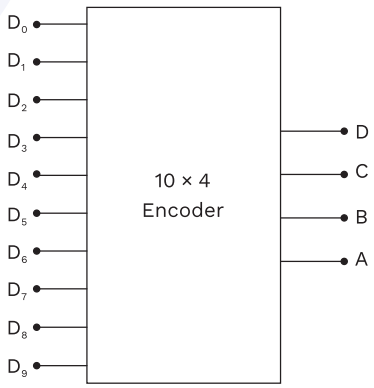


Fig. 5.70 10 X 4 Encoder

The truth table:

Inputs										Outputs			
D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A	B	C	D
0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	0	1	1	0

Inputs										Outputs			
0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0	1	0	0	1

Table 5.25

From the truth table,

$A = D_8 + D_9$

$B = D_4 + D_5 + D_6 + D_7$

$C = D_2 + D_3 + D_6 + D_7$

$D = D_1 + D_3 + D_5 + D_7 + D_9$

Logic diagram of 10 × 4 encoder:

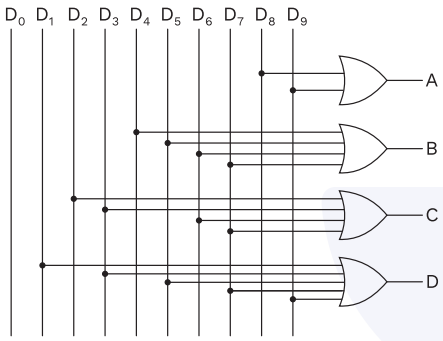


Fig. 5.71 Logic Diagram of 10 X 4 Encoder

Priority Encoder

- It is a simple encoder in which more than one input is high at the same time.
- It works in such a way that if two or more inputs appear at the same time, the one with the highest priority takes precedence.
- Priority encoder is a modified version of the simple encoder, which eliminates the drawback (when more than one input is activated at a time) of simple encoder.

4 × 2 priority encoder:

- In four-bit priority encoder, the inputs are assigned as I_0, I_1, I_2 and I_3 and in addition of output y_1 and y_0 the circuit has another output X.
- X is activated when one or more inputs are 'high' and X = 0 when all the inputs are 'low'.
- When X = 0, then the other outputs y_1 and y_0 are specified as don't care conditions.

The truth table:

I_3	I_2	I_1	I_0	Y_1	Y_0	X
0	0	0	0	×	×	0
0	0	0	1	0	0	1
0	0	1	×	0	1	1
0	1	×	×	1	0	1
1	×	×	×	1	1	1

Table 5.26

In the K-map formation of truth table, at the place of don't care, input may be either logic '0' or '1'.

The truth table:

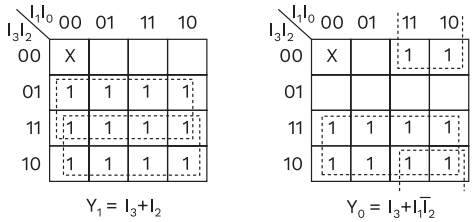


Fig. 5.72 K-map

So, the expression for Y_1, Y_0 and X are,

$$Y_1 = I_2 + I_3$$

$$Y_0 = I_3 + I_1\overline{I}_2$$

$$X = I_0 + I_1 + I_2 + I_3$$

Logic diagram:

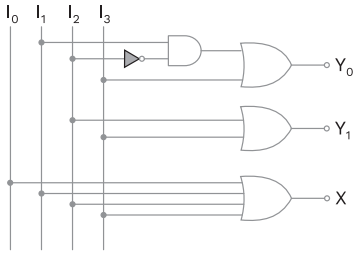


Fig. 5.73 Logic Diagram of Priority Encoder

Chapter Summary

- A decoder with an enable input is the same as a de-multiplexer.
- De-multiplexer is also known as data distributor or series to parallel converter.
- 3 × 8 decoder may work like a 1:8 de-multiplexer and vice versa.
- An encoder has 2^n input lines and n output lines.
- In the priority encoder, if two or more inputs appear simultaneously, the input with the highest priority will take the precedence.
- Priority encoder eliminates the drawback (input selection at a time) of the simple encoder.
- Multiplexer can be used in the communication system, telephone network, waveform generation and many more fields.

Practice Questions

1. What will be the function F realised by the given circuit?

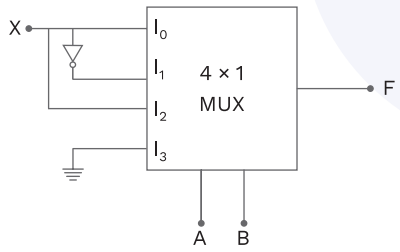


Fig. 5.74

(A) $F = AB\overline{X} + BX + \overline{A}B\overline{X}$

(B) $F = \overline{B}X + \overline{A}B\overline{X}$

(C) $F = B \oplus X$

(D) $F = \overline{B}X + A\overline{B}\overline{X}$

2. What will be the function F realised by the given circuit?

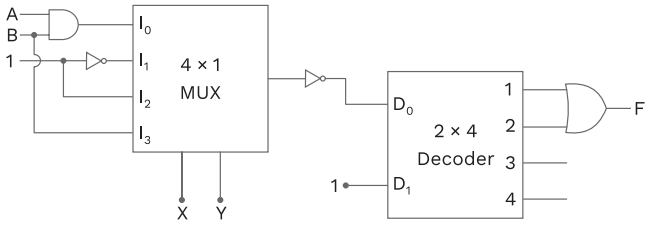


Fig. 5.75

(A) $F = \overline{XY\overline{B} + X\overline{Y}B + \overline{X}\overline{Y}AB}$

(B) $F = XY\overline{A}\overline{B} + \overline{X}Y\overline{A}B + AB$

(C) $F = XYB + X\overline{Y} + \overline{X}\overline{Y}AB$

(D) $F = \overline{X}\overline{Y}AB + \overline{X}Y + AB$

3. The minimum number of 2 × 1 multiplexer required to realize a 64 × 1 multiplexer is _____.

(A) 33

(B) 32

(C) 64

(D) 63

4. For the circuit shown in the figure below:

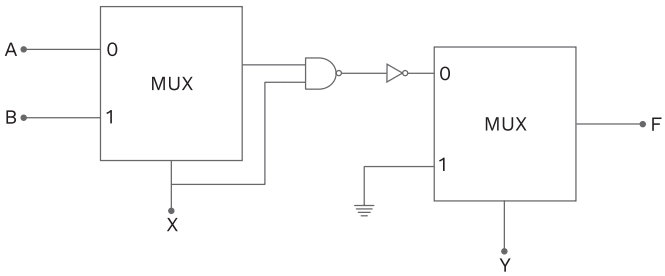


Fig. 5.76

The multiplexer output F is:

(A) $F = \overline{XYB} + XY$

(B) $F = (X \odot B)\, X\overline{Y}$

(C) $F = X\overline{Y}B$

(D) $F = \overline{X}A\overline{Y} + XB\overline{Y}$

5. For the logic circuit's shown below, Boolean function is:

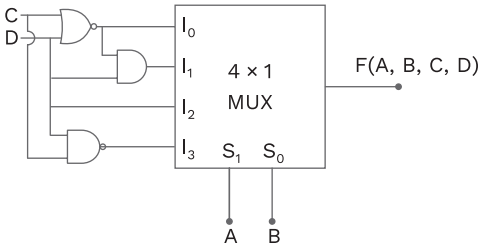


Fig. 5.77

(A) $F = \sum m(0, 9, 11, 12, 13, 14)$

(B) $F = \sum m(0, 1, 5, 9, 14, 15)$

(C) $F = \sum m(1, 2, 3, 5, 7, 15)$

(D) $F = \sum m(0, 9, 12, 14, 15)$

6. A 16K × 16 memory is realized using a single decoder. The minimum number of AND gate required to construct the decoder is:

(A) 2^{16}

(B) 2^{14}

(C) 2^{10}

(D) 2^4

7. For the given circuit, what will be the Boolean function 'F'?

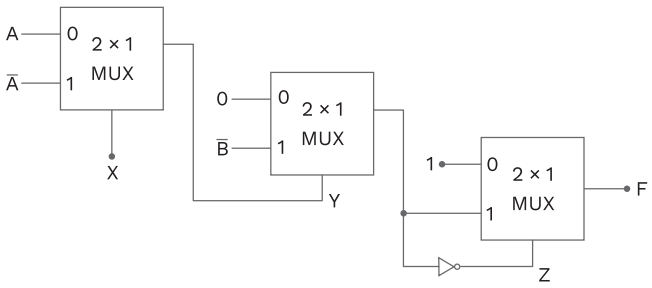


Fig. 5.78

(A) $F = (X \oplus A) \times \overline{B}$

(B) $F = (X \odot A) \times B$

(C) $F = (X \oplus B) \times \overline{A}$

(D) $F = (X \oplus A) + B$

8. What is the function implemented by the given 3–8 decoder?

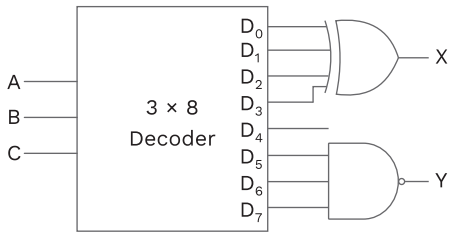


Fig. 5.79

(A) $X = \overline{B}\overline{C}$ and $Y = 1$

(B) $X = \overline{A}$ and $Y = 1$

(C) $X = A$ and $Y = 0$

(D) $X = BC$ and $Y = 0$

9. A one-bit full adder takes 50 ns to generate carry out bit and 100 ns for the sum bit. What is the maximum rate of addition per second where four one-bit full adders are cascaded?

(A) 250 × 109

(B) 25 × 106

(C) 1 × 109

(D) 4 × 106

Answer Keys

1. Solution: (B) **2. Solution: (C)** **3. Solution: (D)** **4. Solution: (C)**

5. Solution: (A) **6. Solution: (B)** **7. Solution: (A)** **8. Solution: (B)**

9. Solution: (D)